CASE STUDY · CREDIT SCORING

케이스 스터디
신용평가 모형 , 처음부터 끝까지

문제 정의 → 탐색 → 결측 처리 → 로지스틱 · 트리 · RF · XGBoost → 튜닝 → AUC · 임계값 · 비용 →
VIP · PDP · SHAP → 고객 세분화 → 최종 보고

지금까지 배운 도구를 한 데이터에 모두 · 데이터 : modeldata::credit_data (n = 4,454) · R: glm · tree · randomForest · xgboost · pROC · pdp · iml · shapviz · cluster

OVERVIEW

학습 목표

1 전체 흐름을 한 번에

문제 정의부터 최종 보고까지 예측모형 프로젝트의 순서와 각 단계에서 내려야 하는 결정을 경험한다 .

2 데이터 다루기

탐색에서 이상한 점과 결측의 의미를 찾고 , 학습 데이터만으로 전처리해 정보 누수를 막는다 .

3 모형 비교와 결정

같은 CV fold 로 여러 모형을 공정하게 비교하고 , AUC 를 넘어 임계값 · 비용 · 보정까지 평가한다 .

4 설명과 활용

VIP · PDP · ICE · SHAP 으로 모형을 설명하고 , 고객 군집과 결합해 최종 모형을 추천한다 .

THE WORKFLOW

예측모형 프로젝트의 10 단계 (오늘의 지도)

1 Part 1
문제 정의
무엇을 , 왜 예측하나

2 Part 1
평가 기준
AUC · 비용 · 해석

3 Part 1
데이터 이해
변수 · 시점 · 누수

4 Part 2
탐색 (EDA)
분포 · 이상치 · 결측

5 Part 3
분할
학습 / 테스트 먼저

6 Part 3
전처리
결측 · 범주 · 파생변수

7 Part 4
모형 적합
기준선 → 복잡한 모형

8 Part 5
평가 · 결정
임계값 · 비용 · 보정

9 Part 6
해석
VIP · PDP · SHAP

10 Part 7–8
활용 · 보고
세분화 · 추천 · 모니터링

실제 프로젝트에서 시간이 가장 많이 드는 곳 : 1~6 단계 . 모형 적합 (7 단계) 은 생각보다 작은 부분이다

PART 1

문제 정의와 데이터

무엇을 예측하는가 , 무엇이 비싼 실수인가 , 어떤 변수를 써도 되는가

PROBLEM

비즈니스 문제 : 이 대출 신청을 승인할까 ?

은행에 대출 신청이 들어온다 . 신청서의 정보로 “나중에 갚지 못할 (bad) 확률”을 예측해 승인 · 거절 · 추가 심사를 결정하고 싶다 .

FN 불량 고객을 승인 (놓침)

대출금 일부를 잃는다 . 손실이 크다 . 실제 bad 인데 good 으로 예측 = False Negative (사건을 bad 로 둘 때)

FP 우량 고객을 거절 (기회 손실)

받을 수 있었던 이자 수익을 잃고 고객도 떠난다 . 손실은 상대적으로 작다 . 실제 good 인데 bad 로 예측 = False Positive

이 강의의 가정

- 반응변수 : Status (bad = 사건 , good). 사건 비율 28%
- 비용 : 불량 승인 1 건 = 5, 우량 거절 1 건 = 1 (5 : 1). 실제 값은 은행의 손실률 · 금리에서 정해진다 → 결론이 이 가정에 따라 달라짐을 기억
- 예측은 신청 시점의 정보만으로 해야 한다

DEFINE SUCCESS FIRST

모형을 만들기 전에 평가 기준부터 정한다

순위를 잘 매기나

AUC

위험한 신청자를 위로 올리는 능력 . 임계값과 무관 → 모형 비교의 주 지표

결정이 싸게 먹히나

기대 비용

임계값을 정한 뒤 (5 · FN + 1 · FP) / n. 실제 업무 성과에 가장 가깝다

확률이 믿을 만한가

보정 · Brier

“ 불량 확률 20%” 가 정말 20% 인가 ? 비용 기반 임계값 · 가격 결정에 필수

설명할 수 있나

해석 가능성

거절 사유를 고객에게 설명해야 하고 , 감독기관 · 내부 검증을 통과해야 한다

정확도 (accuracy) 는 왜 없나 ? 불량이 28% 뿐이라 “전부 good” 이라고만 해도 72% 가 맞는다 . 불균형 자료에서 정확도는 거의 아무것도 말해 주지 않는다 (Part 4 에서 확인)

DATA

데이터 : credit_data (modeldata 패키지)

4,454
대출 신청 건수 (행)

13
설명변수 (수치 9 + 범주 4)

28.2%
불량 (bad) 비율

455
결측이 있는 셀 (6 개 변수)

변수	의미	변수	의미
Seniority	현 직장 근속 연수	Expenses	월 지출
Home	주거 형태 (owner, rent, parents, priv, other, ignore)	Income	월 소득
Time	대출 기간 (개월)	Assets	자산
Age	나이	Debt	부채
Marital	혼인 상태	Amount	대출 신청 금액
Records	과거 연체 · 사고 기록 유무	Price	구매하려는 물건의 가격
Job	고용 형태 (fixed, freelance, partime, others)	Status	반응변수 : good / bad

스페인 은행의 신용 자료 (금액 단위는 원 자료 기준 , 일부 범주 의미는 문서화되어 있지 않음 — ignore, priv 등 → 실무라면 데이터 담당자에게 확인)

BEFORE MODELING

변수 점검 : 시점 · 누수 · 써도 되는 변수인가

1 신청 시점에 아는가 ?

모든 설명변수는 신청서 · 신용조회로 신청 시점에 알 수 있다 . Status 만 나중에 관측된다 . 승인 후의 정보 (연체 횟수 등) 가 섞이면 누수 — bank 데이터의 duration 과 같은 문제

2 반응변수의 정의

“bad” 의 정의 (몇 개월 연체 ?) 와 관측 기간을 확인해야 한다 . 정의가 바뀌면 모형도 바뀐다 . 이 데이터는 정의가 문서에 없다 → 한계로 기록

3 써도 되는 변수인가 ?

나이 · 혼인 상태는 많은 나라에서 신용 결정에 사용이 제한된다 . 예측에 도움이 되어도 법 · 윤리적으로 쓸 수 없을 수 있다 . 수업에서는 모두 쓰되 , 해석에서 다시 논의

숨은 함정 : 이 데이터는 “승인된 사람” 만의 결과다

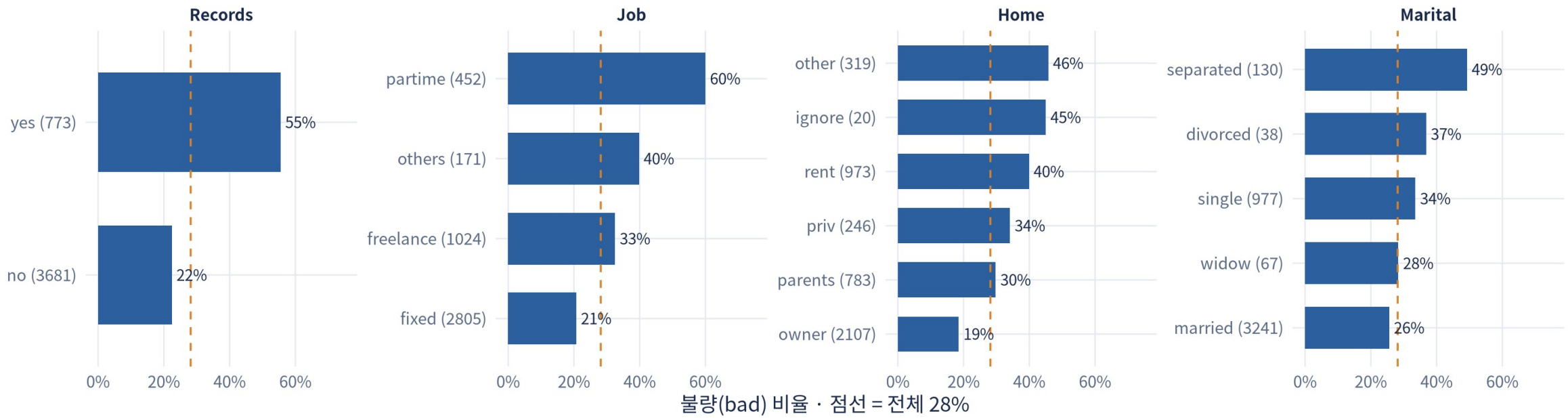
거절된 신청자는 갚았는지 알 수 없어 데이터에 없다 (reject inference 문제). 그래서 모형은 “과거 심사를 통과한 사람들 중에서의” 위험을 배운다 → 새로운 유형의 신청자에게는 성능이 다를 수 있다

PART 2

탐색적 데이터 분석

분포를 보고 , 이상한 점을 찾고 , 결측이 무엇을 의미하는지 묻는다

범주형 변수별 불량률 : 이미 강한 신호가 보인다



불량(bad) 비율 · 점선 = 전체 28%

Records

연체 기록 있음 55% vs 없음 22% → 2.5 배

Job

partime 60%, fixed 21%. 고용 안정성

Home

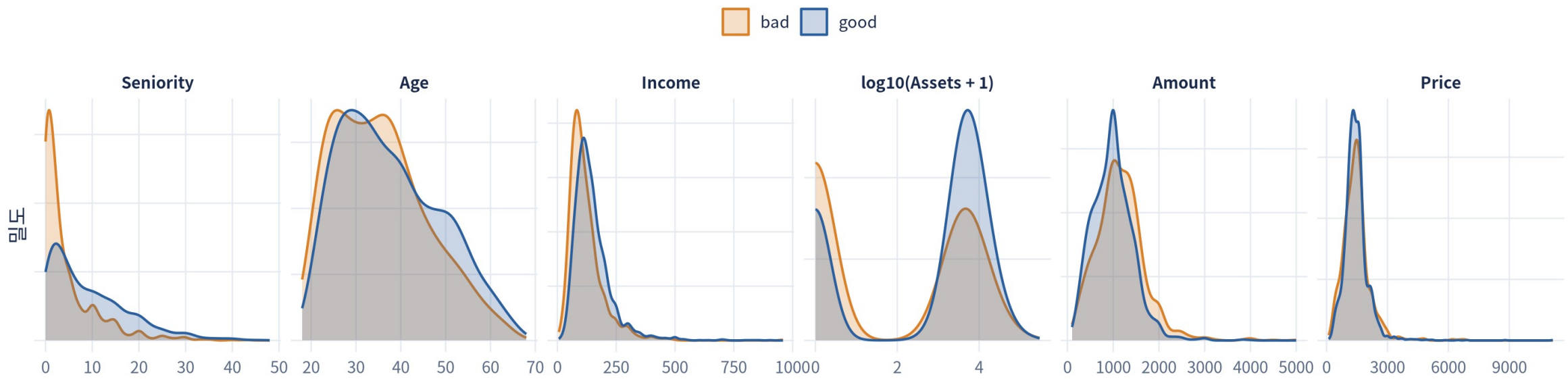
owner 19% vs rent 40%, other 46%

Marital

separated 49%, married 26%. 단 , 표본이 작은 범주 주의 (divorced 38 명)

괄호 안 = 범주의 관측치 수. 전체 데이터 기준 (탐색은 분할 전 전체로 해도 되지만 , 전처리 규칙은 학습 데이터에서만 정한다)

수치형 변수 : 근속 · 소득 · 자산이 두 집단을 가른다



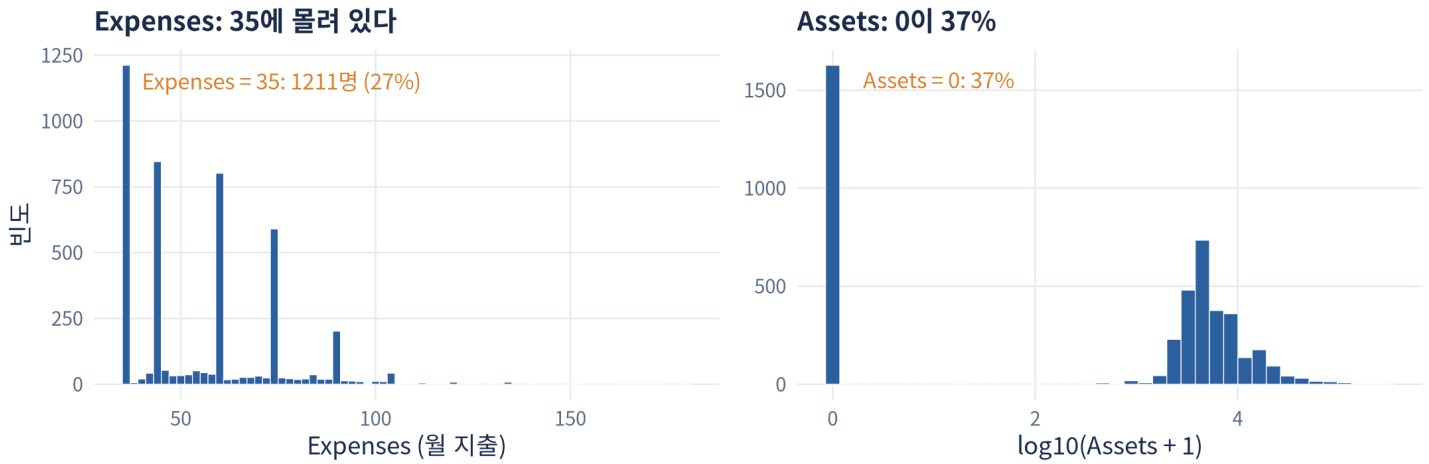
보이는 것

- bad(주황) 는 근속 0~2 년에 몰려 있다
- Assets: 0(왼쪽 봉우리) 에 bad 가 많다
- Age, Amount, Price 는 두 분포가 많이 겹친다

모형에 주는 힌트

- Income · Assets · Price 는 오른쪽 꼬리가 길다 → 로지스틱에는 로그 변환을 고려 , 트리 모형은 상관없음
- 단변량에서 약해도 다른 변수와 함께 쓰면 중요할 수 있다 (예 : Amount 와 Price)

이상한 점 찾기 : 데이터가 만들어진 과정을 의심하라



Expenses = 35 가 27%

35 미만 값이 하나도 없고 35 에 1,211 명이 몰려 있다 . 최소값을 일괄 입력했거나 "미기재 → 35" 같은 규칙이 있었을 가능성 . 진짜 지출이 아니라 코드일 수 있다

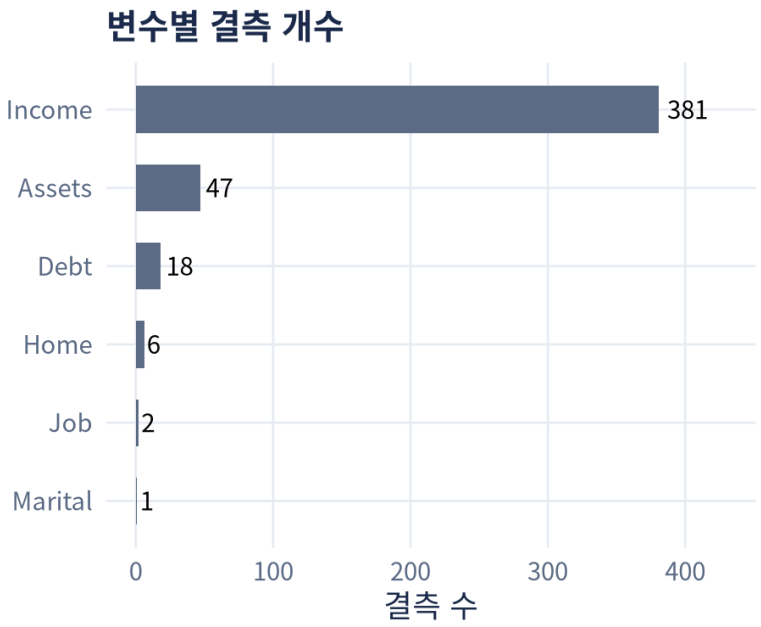
Assets = 0 이 37%

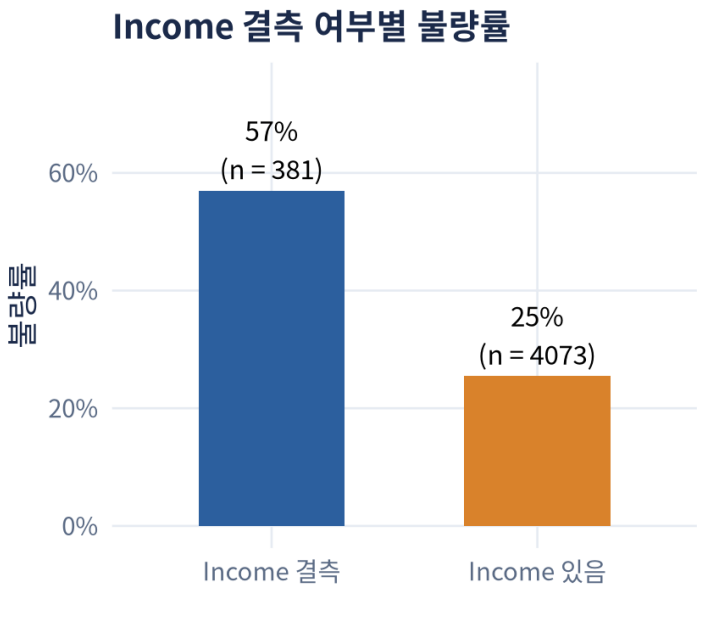
0 인 사람의 불량률 40% vs 양수 21%. " 자산 없음" 일 수도 , " 신고하지 않음" 일 수도 있다 . 의미가 다르면 처리도 달라져야 한다 (0 그대로 vs 결측)

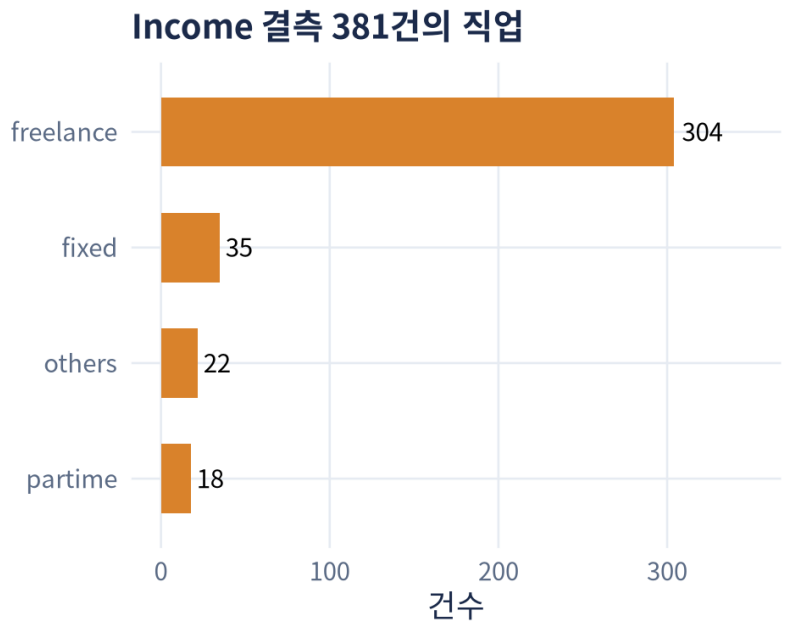
그래서 무엇을 하나 ?

- 데이터를 만든 사람 (현업) 에게 묻는다 → 가장 중요
- 답을 모르면 지시변수 (Expenses_35, Assets_0) 를 만들어 모형이 구분하게 할 수 있다
- 트리 모형은 0 이라는 값 자체로 분할하므로 자동으로 처리되는 편

결측 : 무작위로 빠진 것이 아니다

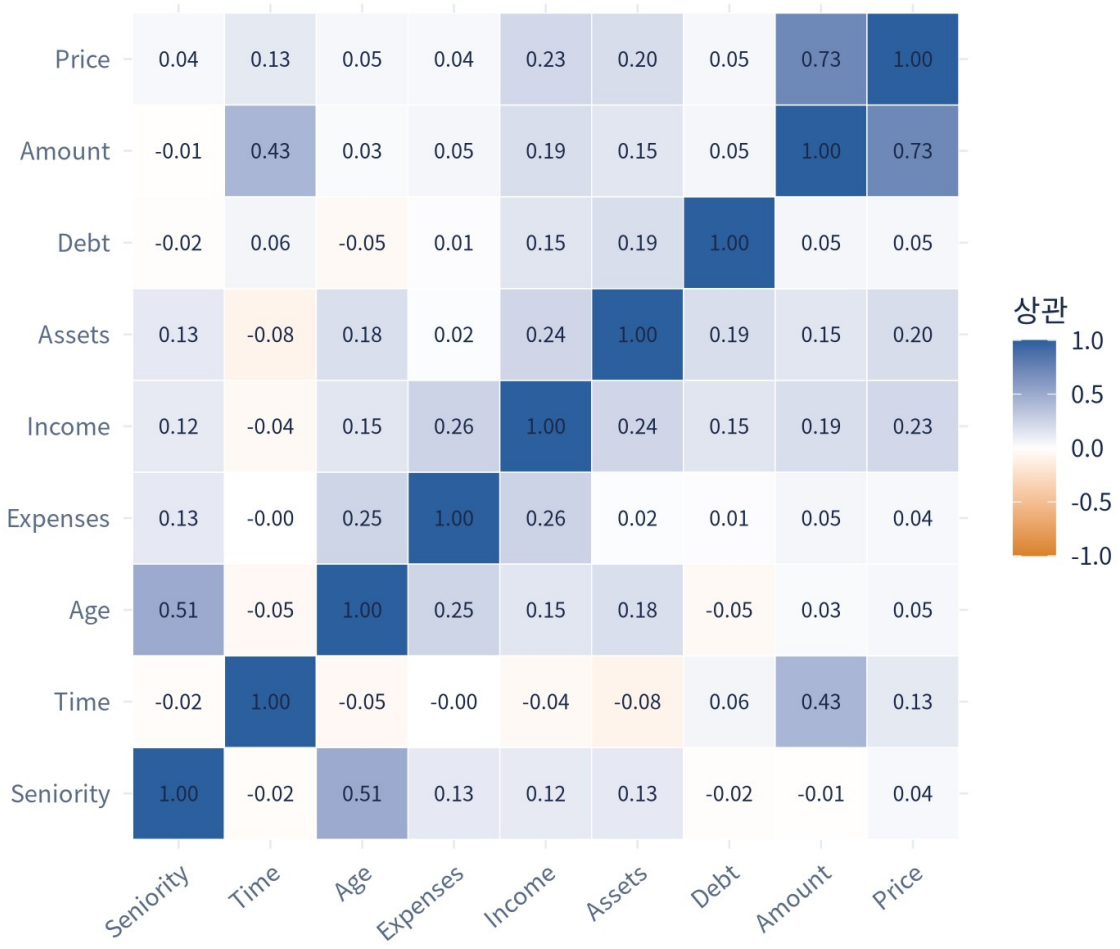






Income 이 비어 있는 사람의 불량률은 57% (있는 사람 25%)
결측의 80%(304 건) 가 freelance. 소득을 증빙하기 어려운 사람일수록 비어 있다 → MNAR(결측 자체가 정보). 이런 결측을 그냥 지우거나 평균으로 채우면 위험 신호를 버리게 된다

변수 간 상관 : 파생변수의 아이디어



Amount – Price: 0.73

비싼 물건일수록 많이 빌린다 . 위험은 금액 자체보다 “물건 값 대비 얼마를 빌리나” → LTV = Amount / Price

Amount – Time: 0.43

많이 빌리면 길게 갚는다 . 월 상환액 대비 소득이 부담 → Burden = (Amount / Time) / Income

Seniority – Age: 0.51

나이가 많으면 근속도 길다 . 해석할 때 둘의 효과가 섞인다 (PDP · 중요도에서 다시 확인)

PART 3

분할과 전처리

테스트 데이터는 마지막까지 봉인 — 모든 규칙은 학습 데이터에서 배운다

SPLIT FIRST

먼저 나누고 , 그다음 전처리한다

전체 4,454 건

→

학습 3,118 (70%) 불량 28.16%

테스트 1,336 (30%) 불량 28.14%

층화 (stratified) 추출 : good 과 bad 안에서 각각 70% 씩 → 두 데이터의 불량률이 같다

1 잘못된 순서

전체 데이터의 중앙값으로 결측을 채운 뒤 분할 → 테스트 데이터의 정보가 학습에 새어 들어간다 (누수). 효과는 작아 보여도 kNN 대체 · 표준화 · 변수 선택이면 커진다

2 올바른 순서

분할 → 학습 데이터로 대체값 · 범주 정리 규칙 계산 (prep_fit) → 같은 규칙을 학습 · 테스트에 적용 (prep_apply). CV 안에서도 원칙은 같다

테스트 데이터의 역할 : 모형 선택 · 튜닝 · 임계값 결정이 모두 끝난 뒤 단 한 번 성능을 확인하는 것 . 그 전에는 열어 보지 않는다

MISSING DATA STRATEGY

결측 처리 방법 비교

방법	장점	단점	이 데이터에서
완전한 행만 사용	간단	정보 손실 , 남은 데이터가 편향	학습 3,118 중 2,834 만 남고 , 불량률 57% 인 집단이 빠진다
평균 · 중앙값 대체	간단 , 행 유지	“ 결측이었다” 는 정보가 사라짐 , 분산 축소	Income 결측자가 평범한 소득자로 둔갑
대체 + 결측 지시변수	결측 자체의 정보를 보존	변수가 늘어남	Income_NA = yes / no 추가 → 채택
kNN · 모형 기반 대체	다른 변수로 그럴듯하게 채움	계산량 , 결측이 정보일 땐 여전히 손실	Chapter 2 의 knnImputation
모형이 직접 처리	전처리 불필요	모형에 따라 지원 여부 다름	XGBoost 는 NA 를 그대로 받아 방향을 학습

범주형 결측 (Home 6, Marital 1, Job 2 건): 학습 데이터의 최빈값으로 대체 . Home 의 ignore(20 건) 는 의미가 불분명하고 작아 other 와 합침

수치형 결측 (Income, Assets, Debt) 은 학습 데이터 중앙값으로 대체 . 대체값은 학습 데이터에서만 계산

DOES IT MATTER?

결측 지시변수와 파생변수 , 실제로 도움이 되었나 ?

전처리 (로지스틱 회귀)	5-fold CV AUC	차이
중앙값 대체만	0.8310	기준
중앙값 + 결측 지시변수 (Income_NA)	0.8366	+0.0056
↳ **파생변수 (LTV, Burden) 제외**	0.8355	+0.0045

읽는 법

- 지시변수 : 작지만 분명한 개선 . 로지스틱에서 Income_NA 의 오즈비 3.6 (Part 6)
- 파생변수 : 거의 차이 없음 (+0.001). CV 표준오차 (약 0.007) 보다 훨씬 작다

교훈 : 효과가 없었던 시도도 보고한다

LTV 와 Burden 은 예측력은 거의 더하지 않았지만 , " 물건 값 대비 대출 비율" 처럼 현업이 이해하는 말로 설명할 수 있어 남겨 두었다 .
모든 비교는 같은 5 개 fold 에서 했다 . 다른 fold 면 0.005 정도의 차이는 쉽게 뒤집힐 수 있다 → 작은 차이에 큰 의미를 두지 않는다

R: 학습 데이터에서 규칙을 배우고 , 똑같이 적용한다

R CONSOLE

```
library(modeldata); data(credit_data)
d <- credit_data
d$Status <- relevel(d$Status, ref = "good")   # bad = 사건

# 층화 분할 70 / 30
set.seed(2026)
idx <- unlist(lapply(split(seq_len(nrow(d)), d$Status),
                     function(i) sample(i, round(0.7 * length(i)))))
train.raw <- d[idx, ];  test.raw <- d[-idx, ]

# 1) 규칙 배우기 : 학습 데이터만 사용
prep_fit <- function(tr) list(
  med  = sapply(tr[, c("Income","Assets","Debt")], median, na.rm = TRUE),
  mode = sapply(tr[, c("Home","Marital","Job")],
                function(x) names(which.max(table(x)))))
P <- prep_fit(train.raw)
```

R CONSOLE

```
# 2) 같은 규칙 적용
prep_apply <- function(df, p) {
  df$Income_NA <- factor(
    ifelse(is.na(df$Income),
           "yes", "no"))
  for (v in names(p$med))
    df[[v]][is.na(df[[v]])] <-
      p$med[v]
  # ( 범주 최빈값 대체 ,
  #  ignore -> other  생략 )
  df$LTV    <- df$Amount / df$Price
  df$Burden <- (df$Amount /
       df$Time) / df$Income
  df
}
train <- prep_apply(train.raw, P)
test  <- prep_apply(test.raw,  P)
```

PART 4

모형 적합과 비교

기준선에서 시작해 로지스틱 → 트리 → Random Forest → XGBoost, 같은 fold 로 공정하게

EXPERIMENT DESIGN

비교 설계 : 모든 모형이 같은 조건에서 경쟁한다

1 같은 fold

학습 데이터를 층화 5-fold 로 한 번 나누고 모든 모형 · 모든 튜닝 후보가 같은 fold 를 쓴다 → fold 운의 차이 제거

2 튜닝도 CV 안에서

RF 의 mtry, XGBoost 의 depth · nrounds, 트리 크기 모두 학습 데이터의 CV 로 결정 . 테스트는 사용하지 않음

3 지표 : AUC

임계값과 무관하게 순위 능력을 비교 . fold 별 AUC 의 퍼짐 (표준편차) 도 함께 본다

4 마지막에 한 번

최종 후보를 학습 데이터 전체로 다시 적합 → 테스트 AUC, 신뢰구간 , DeLong 검정

후보 : 로지스틱 (전체 · stepAIC) · 단일 트리 (tree + cv.tree) · Random Forest · XGBoost

BASELINE

기준선과 정확도의 함정

71.9%

“ 모두 good” 이라고 예측했을 때 테스트 정확도

80.3%

XGBoost, 임계값 0.5 의 정확도

53.5%

같은 모형의 민감도 : 불량 376 명 중 201 명만 잡음

XGBoost, 임계값 0.5	실제 bad	실제 good
예측 bad (거절)	TP 201	FP 88
예측 good (승인)	FN 175	TN 872

무엇이 문제인가

정확도 80% 라도 불량 고객의 47%(175 명) 를 승인한다 . 비용 5 : 1 이면 이 175 건이 가장 비싸다

0.5 는 아무 근거 없는 임계값이다

0.5 는 두 오류의 비용이 같을 때의 기준 . 비용이 다르면 다른 임계값이 맞다 → Part 5. 모형 비교는 임계값과 무관한 AUC 로 한다

MODEL 1 · LOGISTIC

로지스틱 회귀 : 강력한 기준선

R CONSOLE

```
glm.fit  <- glm(Status ~ ., data = train,
                family = binomial)
step.fit <- MASS::stepAIC(glm.fit, trace = 0)
attr(terms(step.fit), "term.labels")   # 남은 변수
p.glm <- predict(glm.fit, test, type = "response")
```

	CV AUC	테스트 AUC
로지스틱 (전체 16 개 변수)	0.837	0.844
stepAIC (Time, Price 제거)	0.836	0.844

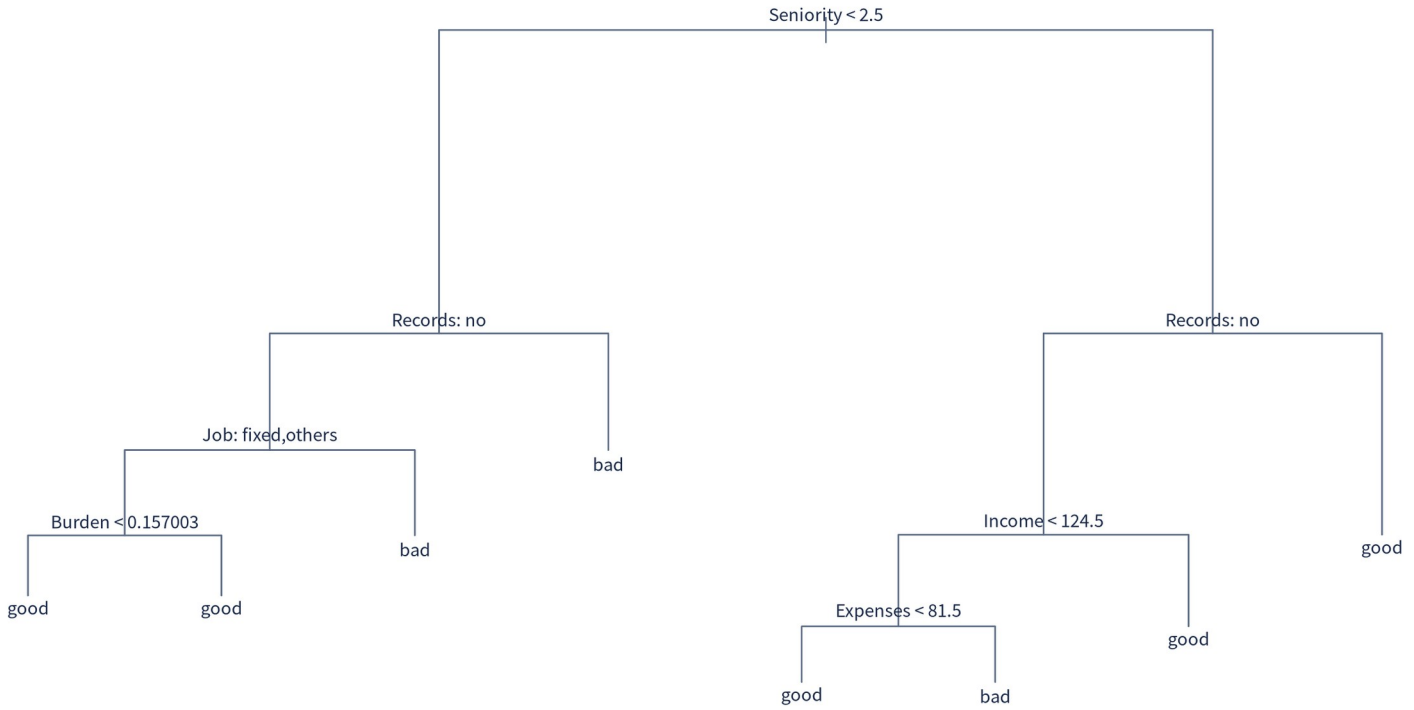
왜 기준선으로 강한가

- n = 3,118, 변수 16 개 : 모수 추정이 안정적
- 주요 관계가 단조 (근속 ↑ → 위험 ↓) 라 선형 로짓으로 잘 근사
- 계수 = 오즈비 → 설명이 쉽다

stepAIC 결과

Time 과 Price 만 빠지고 성능은 그대로 . Price 는 LTV(= Amount/Price) 에 이미 들어 있어서 따로 필요 없다

단일 트리 : 가장 읽기 쉽지만 가장 약하다



cv.tree → 잎 8 개

- 첫 분할 : Seniority < 2.5 (근속 3 년 미만)
- 근속 짧고 연체 기록 있음 : 불량 76%
- 근속 길고 기록 없고 Income > 124.5: 불량 7%
- CV AUC 0.764, 테스트 0.753 — 다른 모형보다 0.08 낮다

쓸모 : 성능은 낮아도 “어떤 변수가 먼저 갈라지는가”를 한 장으로 보여 준다 . 탐색 · 소통용으로 좋고 , 첫 분할 변수들 (Seniority, Records) 은 뒤의 VIP 와도 일치한다

Random Forest: mtry 튜닝

mtry	2	3	4	6	8
5-fold CV AUC	0.843	0.841	0.840	0.837	0.836

R CONSOLE

```
set.seed(1)
rf.fit <- randomForest(Status ~ ., data = train, ntree = 500,
                       mtry = 2, importance = TRUE)
p.rf <- predict(rf.fit, test, type = "prob")[, "bad"]   # 확률
```

읽기

- 작은 mtry 가 가장 좋다 (기본값 floor(√16) = 4 보다 2)
- 변수 간 정보가 겹쳐 다양성 (낮은 ρ) 의 이득이 큼
- 테스트 AUC 0.842

분류에서 확률 예측

type = "prob" 는 트리들의 투표 비율 . 기본 predict() 는 0.5 기준의 클래스만 준다 → AUC · 임계값 분석에는 반드시 확률을 사용

XGBoost: 깊이별로 조기 종료

max_depth	CV 가 고른 nrounds	CV AUC	fold 표준편차
2	361	0.8483	0.016
3	149	0.8458	0.015
4	158	0.8451	0.015
6	93	0.8436	0.016

참고 : 다른 튜닝 방법

- eta, subsample, min_child_weight 까지 Bayesian optimization (지난 튜닝 강의)
- 여기서는 깊이만 바꿔도 차이가 0.005 이내 → 더 정교한 튜닝의 이득은 작을 것

R CONSOLE

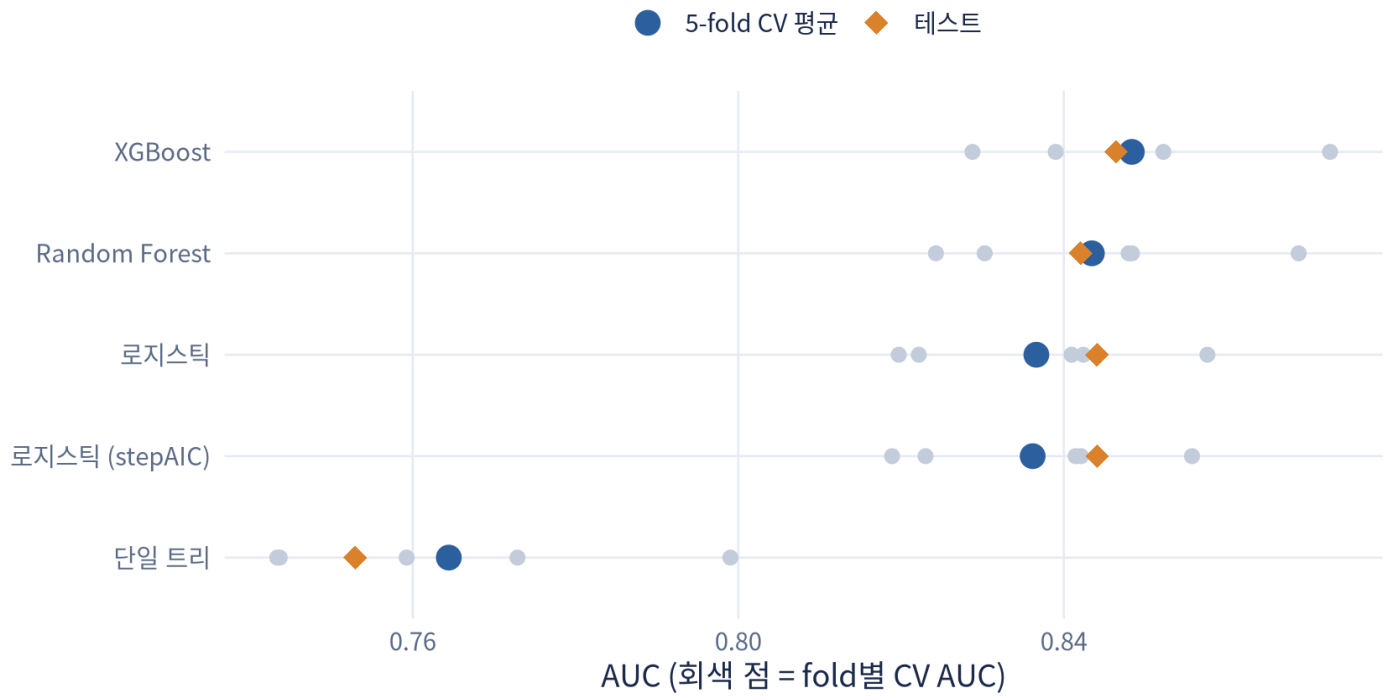
```
cv <- xgb.cv(params = list(
        objective = "binary:logistic",
        eval_metric = "auc", eta = 0.05,
        max_depth = 2, subsample = 0.8,
        colsample_bytree = 0.8),
      data = dtr, nrounds = 3000,
      folds = fold.list,     # 같은 fold!
      early_stopping_rounds = 100)
```

깊이 2 가 가장 좋다

깊이 2 = 두 변수까지의 상호작용만 . 관계가 거의 가법적이라는 신호 → 로지스틱이 강했던 이유와 같은 이야기

COMPARISON

모형 비교 : 상위 세 모형은 거의 같다

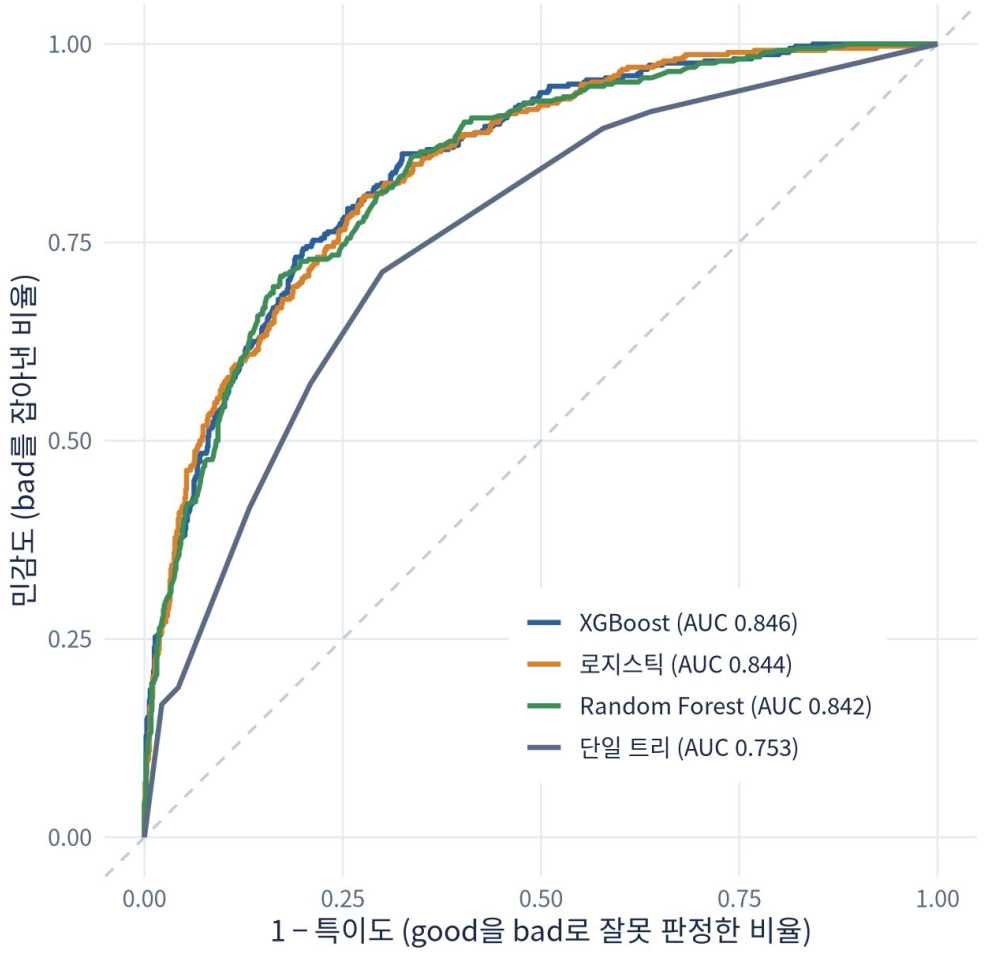


모형	CV AUC	테스트 AUC
XGBoost	0.848	0.846
Random Forest	0.843	0.842
로지스틱	0.837	0.844
stepAIC	0.836	0.844
단일 트리	0.764	0.753

XGBoost 는 CV 의 5 개 fold 모두에서 로지스틱보다 높았다 (평균 +0.011)
작지만 일관된 차이 . 그러나 fold 간 퍼짐 (±0.016) 과 비슷한 크기라 , 테스트에서는 차이가 0.002 로 줄었다 → 다음 슬라이드에서 검정

TEST SET

테스트 데이터에서 단 한 번 : ROC 와 검정



모형	테스트 AUC	95% 신뢰구간
XGBoost	0.846	0.824 – 0.869
로지스틱	0.844	0.822 – 0.867
Random Forest	0.842	0.819 – 0.865
단일 트리	0.753	0.725 – 0.781

DeLong 검정 (같은 테스트 데이터의 두 AUC 비교)

- XGBoost vs 로지스틱 : p = 0.67
- XGBoost vs Random Forest: p = 0.34
- 신뢰구간 폭이 약 ±0.022 → 0.002 차이는 구분할 수 없다
- R: pROC::roc.test(roc1, roc2)

R: 같은 fold 로 여러 모형의 CV 예측 만들기

R CONSOLE

```
library(pROC)
set.seed(1)                          #  층화  5-fold,  한 번만
folds <- integer(nrow(train))
for (cl in levels(train$Status)) {
  i <- which(train$Status == cl)
  folds[i] <- sample(rep(1:5, length.out = length(i)))
}
cvpred <- function(fitfun) {         #  모든 모형이 같은  fold
  pr <- numeric(nrow(train))
  for (f in 1:5)
    pr[folds == f] <- fitfun(train[folds != f, ],
                             train[folds == f, ])
  pr
}
f_glm <- function(tr, te)
  predict(glm(Status ~ ., data = tr, family = binomial),
          te, type = "response")
auc(train$Status, cvpred(f_glm))  # 0.837
```

fitfun(tr, te)

학습 데이터로 적합하고 te 의 P(bad) 를 돌려주는 함수

cvpred()

n 개 관측치 모두의 “보지 않은 모형” 의 예측

CV 예측의 재활용

임계값 결정 (Part 5) 도 이 예측으로 → 테스트를 아낀다

tree · cv.tree 주의

함수 안에서는 do.call("tree", …) 로 호출해야 cv.tree 가 데이터를 찾는다

PART 5

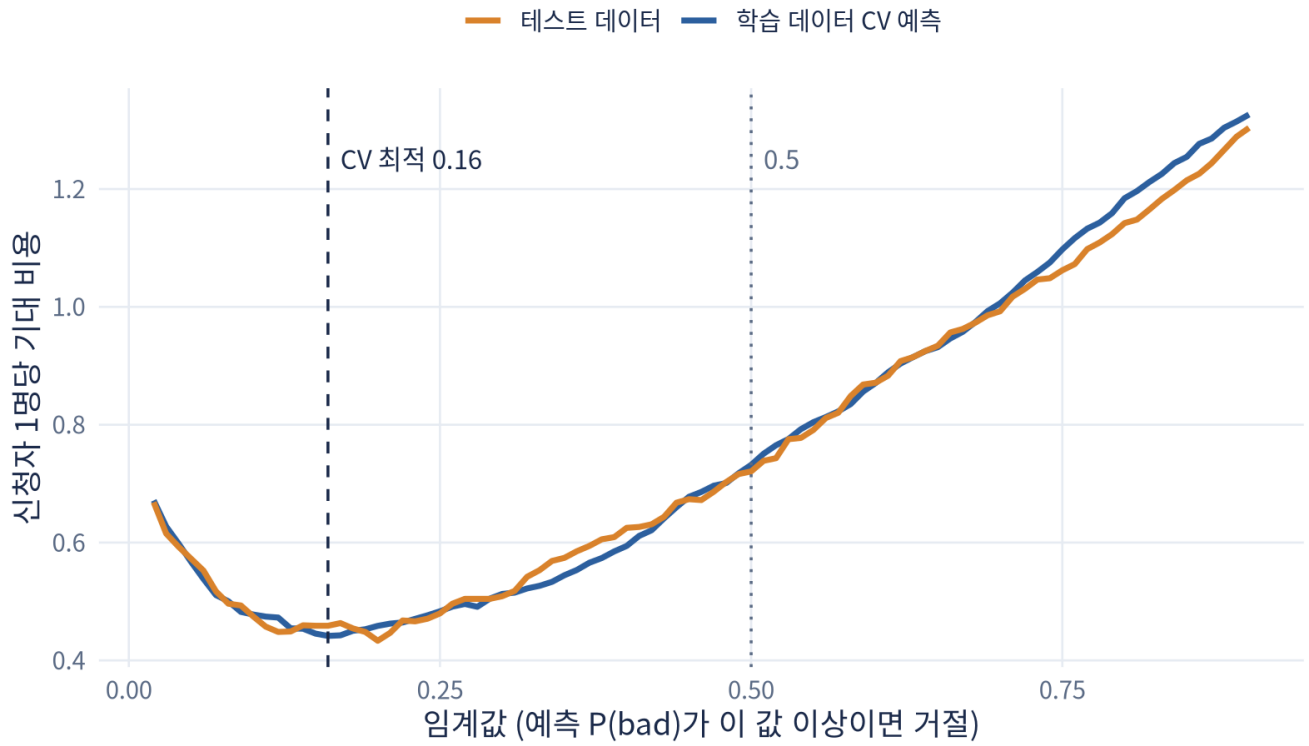
확률을 결정으로

임계값 · 비용 · 이득 곡선 · 보정 — AUC 만으로는 끝나지 않는다

COST-BASED THRESHOLD

비용으로 임계값 정하기

기대 비용 $= (5 \cdot \mathrm{FN} + 1 \cdot \mathrm{FP}) \,/\, \mathrm{n},$ 보정된 확률이면 최적 임계값 $= \mathrm{c_FP} \,/\, (\mathrm{c_FP} + \mathrm{c_FN}) = 1/6$



왜 1/6 인가

P(bad) = p 인 신청자를 승인하면 기대 손실 5p, 거절하면 1 · (1 − p). 5p > 1 − p, 즉 p > 1/6 이면 거절이 싸다

CV 예측으로 고른 값 : 0.16

이론값 1/6 과 거의 같다 → XGBoost 의 확률이 잘 보정되어 있다는 뜻 . 학습 CV 와 테스트 곡선의 모양도 비슷

BUSINESS IMPACT

임계값에 따라 비용이 얼마나 달라지나 (테스트)

결정 규칙	XGBoost	로지스틱
모두 승인	1.407	1.407
임계값 0.5	0.721	0.714
임계값 1/6 (이론)	0.454	0.463
CV 로 고른 임계값 (0.16 / 0.19)	0.459	0.460

신청자 1 명당 기대 비용 (불량 승인 = 5, 우량 거절 = 1)

XGBoost, 임계값 0.16	실제 bad	실제 good
거절	TP 334	FP 403
승인	FN 42	TN 557

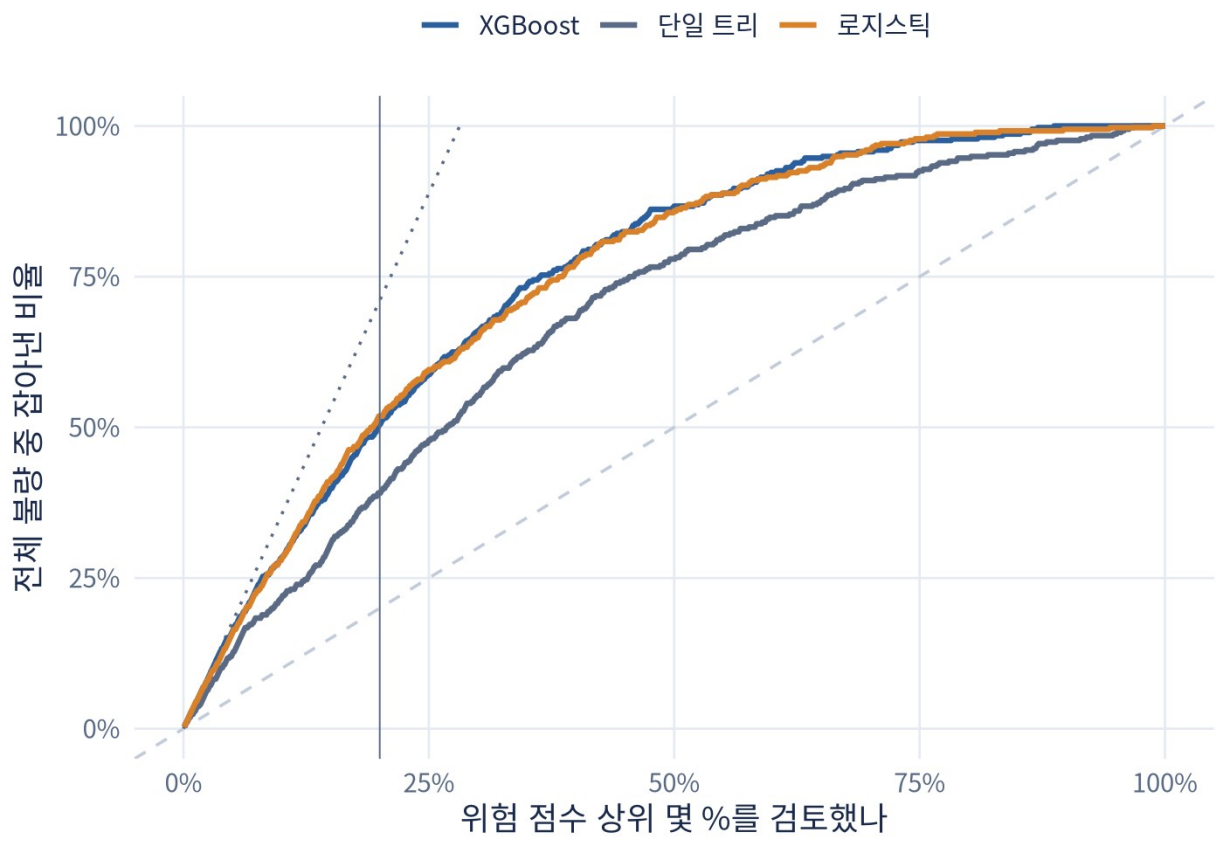
불량의 89% 를 거절

대신 신청자의 55% 를 거절한다 . 우량 고객 403 명도 함께 거절

임계값을 0.5 에서 0.16 으로 바꾸는 것만으로 비용이 36% 줄었다
모형을 XGBoost 로 바꾼 효과 (로지스틱 대비) 보다 훨씬 크다 . 실무에서 가장 큰 개선은 종종 “결정 규칙” 에서 나온다 .
단 , 55% 거절은 영업 목표와 충돌할 수 있다 → 실제로는 “승인율 ⩾ 60%” 같은 제약 아래 비용을 최소화하거나 , 중간 구간은 사람이 심사한다

GAINS CHART

누적 이득 곡선 : 위험 점수 상위부터 심사하면



상위 몇 % 를 검토	XGBoost	로지스틱	무작위
10%	28%	28%	10%
20%	51%	52%	20%
30%	66%	65%	30%

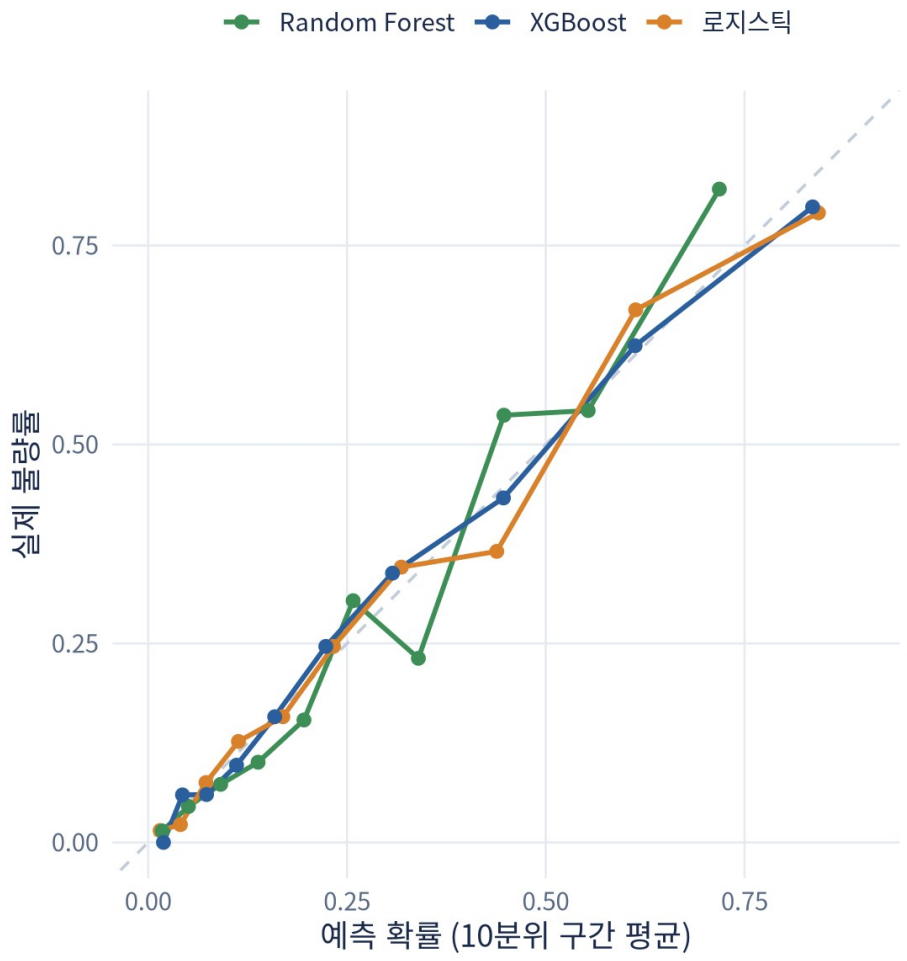
읽는 법

- 점선 (대각) = 무작위로 고른 경우 , 점 (왼쪽 위 끝까지 직선) = 완벽한 모형
- 심사 인력이 신청서의 20% 만 볼 수 있다면 불량의 절반을 그 안에서 만난다
- 이 그림에서도 두 모형은 구분되지 않는다

Lift = (상위 q% 에서의 불량 포착률) / q. 상위 10% 의 lift ≈ 2.8 (홈페이지의 Lift Chart 자료 참고)

CALIBRATION

보정 : “20%” 라고 말하면 정말 20% 인가 ?



모형	Brier 점수	보정
XGBoost	0.1368	좋음
로지스틱	0.1369	좋음
Random Forest	0.1393	들쭉날쭉
단일 트리	0.1671	잎 8 개 → 확률 8 개뿐

왜 중요한가

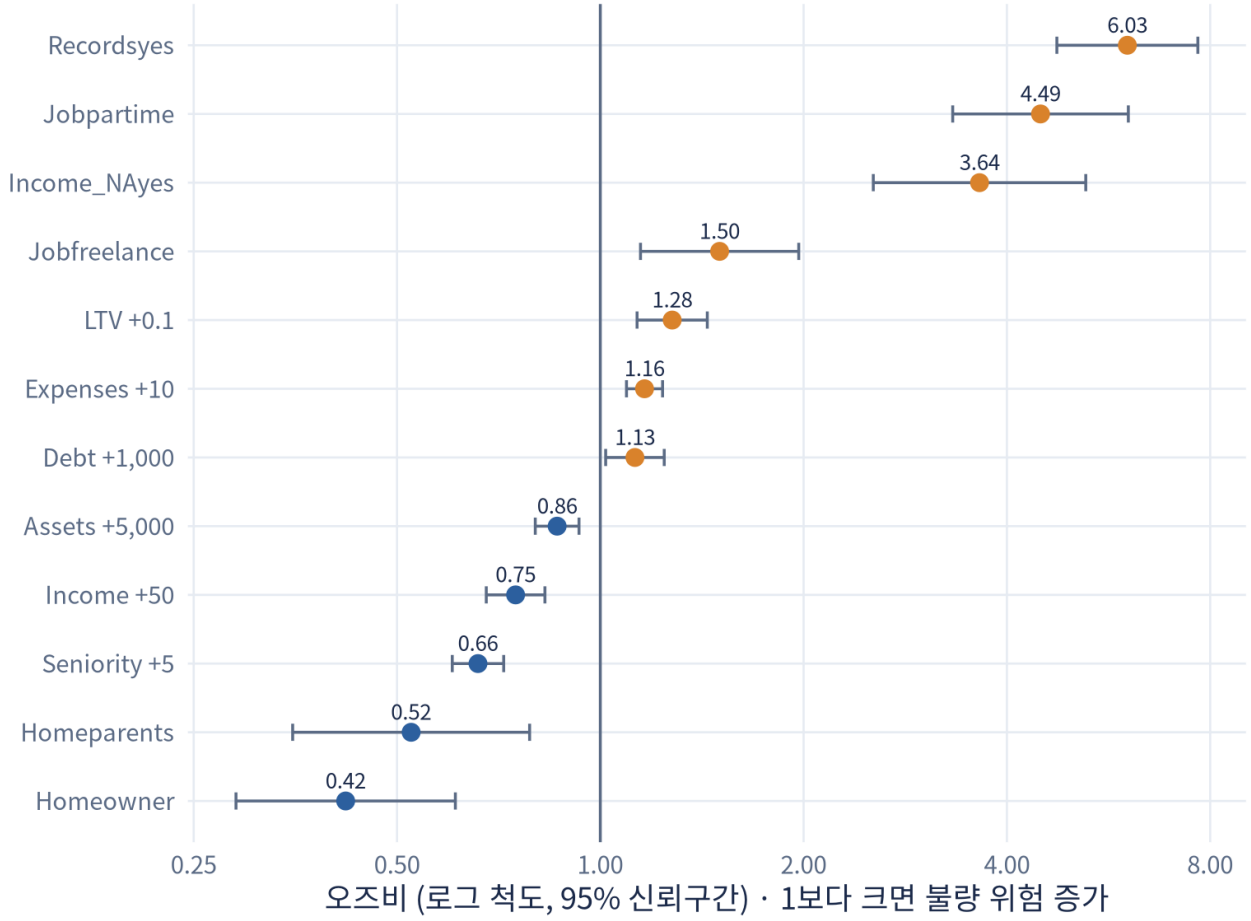
- 비용 기반 임계값 1/6 은 확률이 보정되어 있을 때만 맞다
- 금리 · 한도를 확률에 따라 정할 때 (위험 기반 가격) 확률의 크기 자체가 쓰인다
- Brier = 평균 $(p - y)^2$: 순위와 보정을 함께 보는 지표 . 보정이 나쁘면 Platt scaling · isotonic 회귀로 사후 보정

PART 6

모형 해석

무엇이 위험을 높이는가 — 로지스틱 오즈비 , VIP, PDP, ICE, SHAP, 그리고 개별 신청자 설명

로지스틱 회귀의 오즈비 : 가장 직접적인 해석



위험을 높이는 것

- 연체 기록 : 오즈 6.0 배
- partime: fixed 대비 4.5 배 · freelance 1.5 배
- Income 결측 : 3.6 배
- LTV 0.1 증가마다 1.28 배

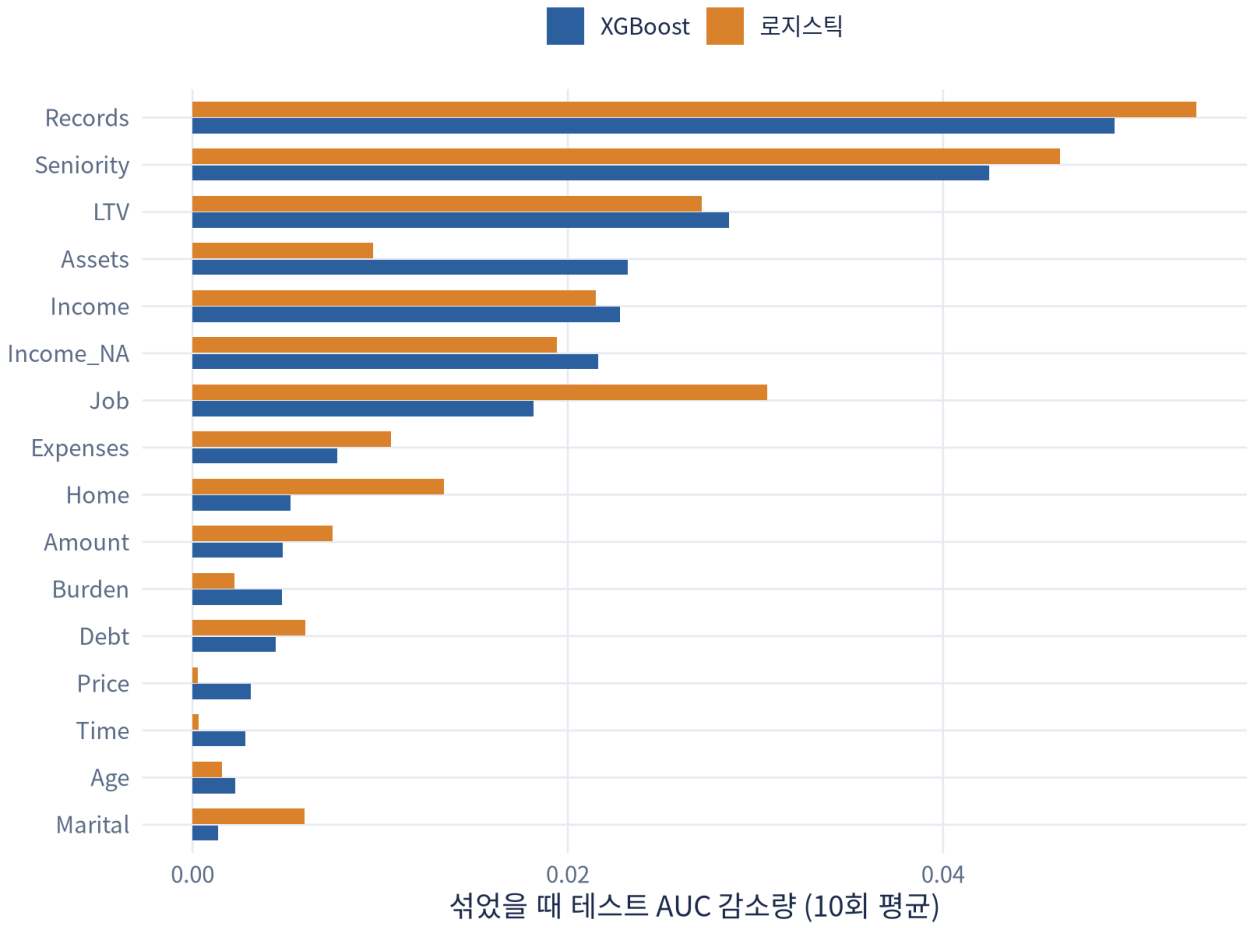
위험을 낮추는 것

- 근속 5 년마다 0.66 배
- 소득 50 증가마다 0.75 배
- 주택 소유 : other 대비 0.42 배

p < 0.05 인 항만 표시 . 수치형은 의미 있는 단위로 환산 (예 : 근속 +5 년). 범주형 기준 범주 : Home = other, Job = fixed, Marital = divorced, Records = no

VIP

순열 중요도 (AUC 감소): 두 모형이 보는 것은 비슷하다



공통 상위 : Records, Seniority, LTV

Records 를 섞으면 XGBoost 의 테스트 AUC 가 0.049 떨어진다 . 둘 다 1, 2 위가 같다

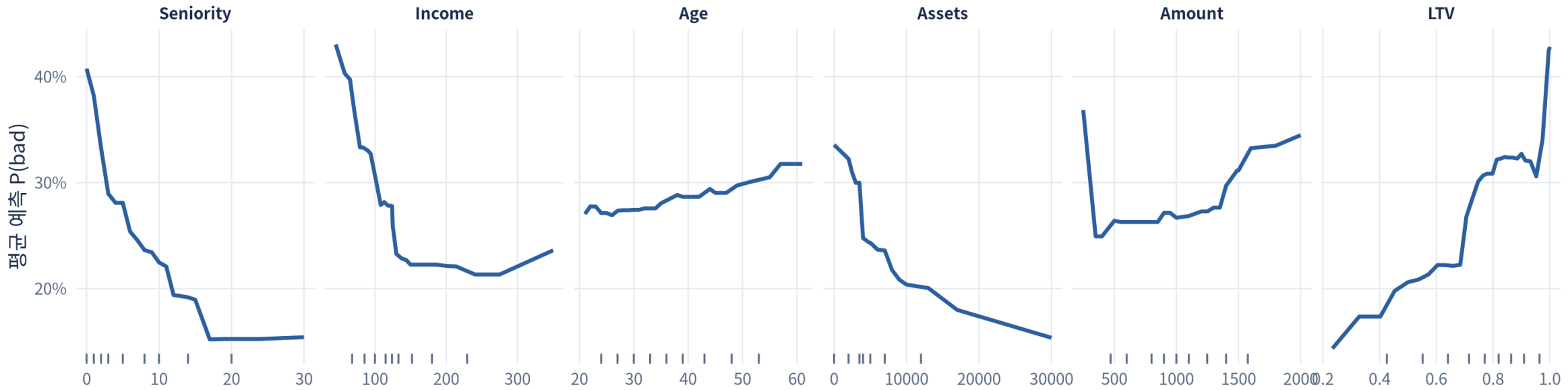
다른 점

- Job: 로지스틱이 더 의존 (0.031 vs 0.018)
- Assets: XGBoost 가 더 의존 (0.023 vs 0.010) — 0 과 양수의 차이 같은 비선형을 트리가 잡음
- Age, Time, Price: 둘 다 거의 0

지표를 AUC 로 바꾼 순열 중요도는 직접 구현 (변수를 10 번 섞어 평균). Income 과 Income_NA, Amount · Price 와 LTV 처럼 연결된 변수는 중요도가 나뉜다

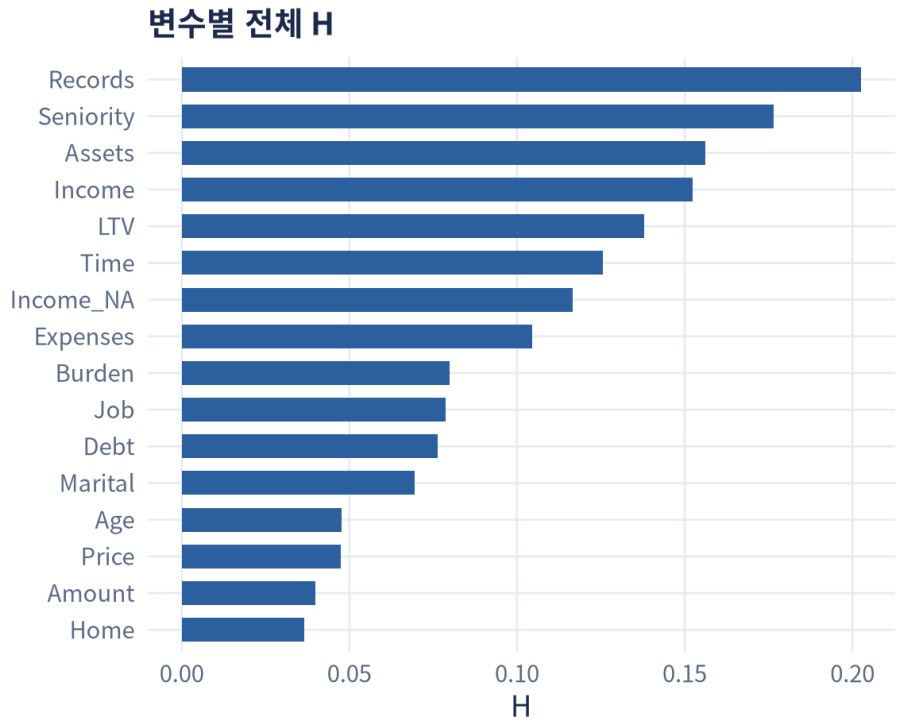
PDP

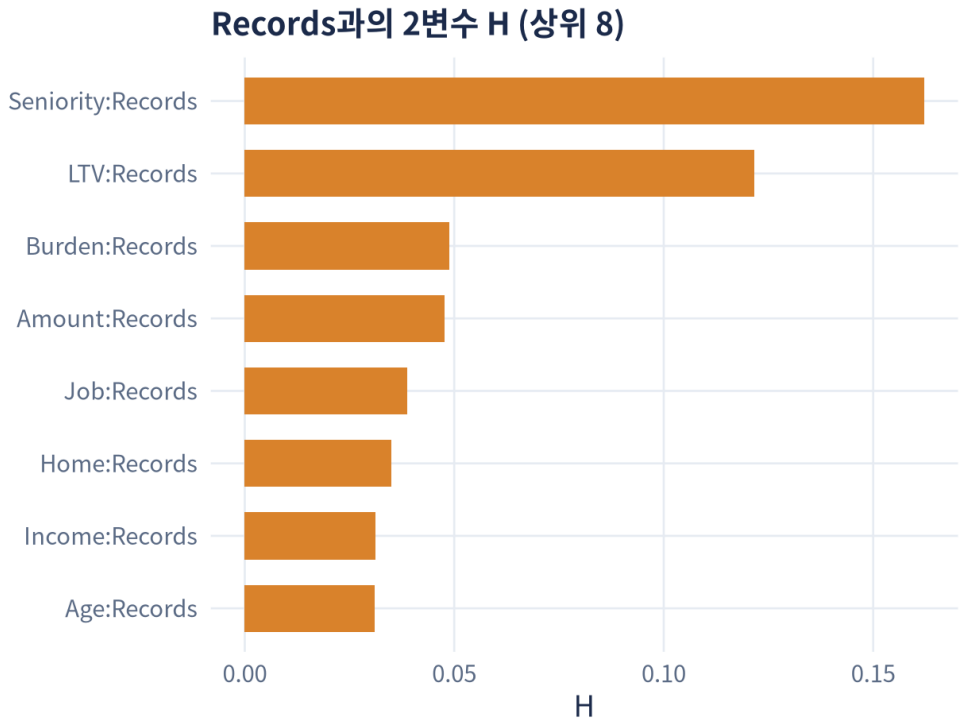
부분 의존도 (XGBoost, 확률 척도)



Seniority	0 년 41% → 17 년 이후 15% 로 평평	**Income**	~120 까지 급감 , 그 이상은 거의 평평	**Age**	근속을 고정하면 오히려 약간 증가 (상관된 변수 주의)
Assets	0 에서 34% → 자산이 많을수록 감소	**Amount**	U 자 모양 : LTV 를 고정한 채 바꾼 효과라 해석 어려움	**LTV**	0.9 를 넘으면 급증 (물건 값 대부분을 빌림)

상호작용의 크기 : H- 통계량



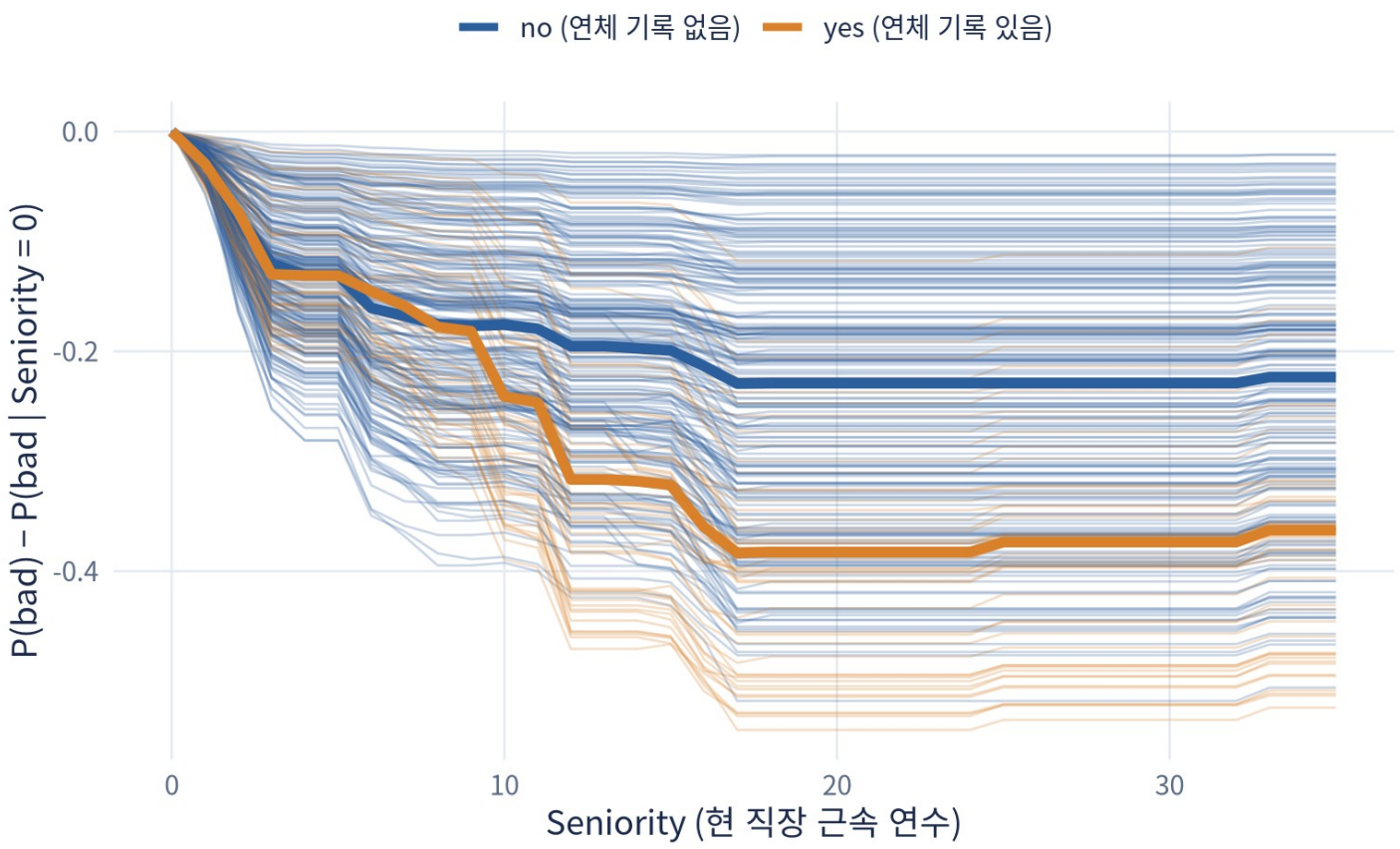


전체 H 최대 : Records 0.20, Seniority 0.18

짝 : Seniority × Records 0.16, LTV × Records 0.12

iml::Interaction, 학습 데이터 300 명 표본 → 값은 표본에 따라 흔들리므로 순위 위주로 읽는다

c-ICE: 연체 기록이 있으면 근속의 효과가 더 크다



근속 0 → 20 년의 위험 변화

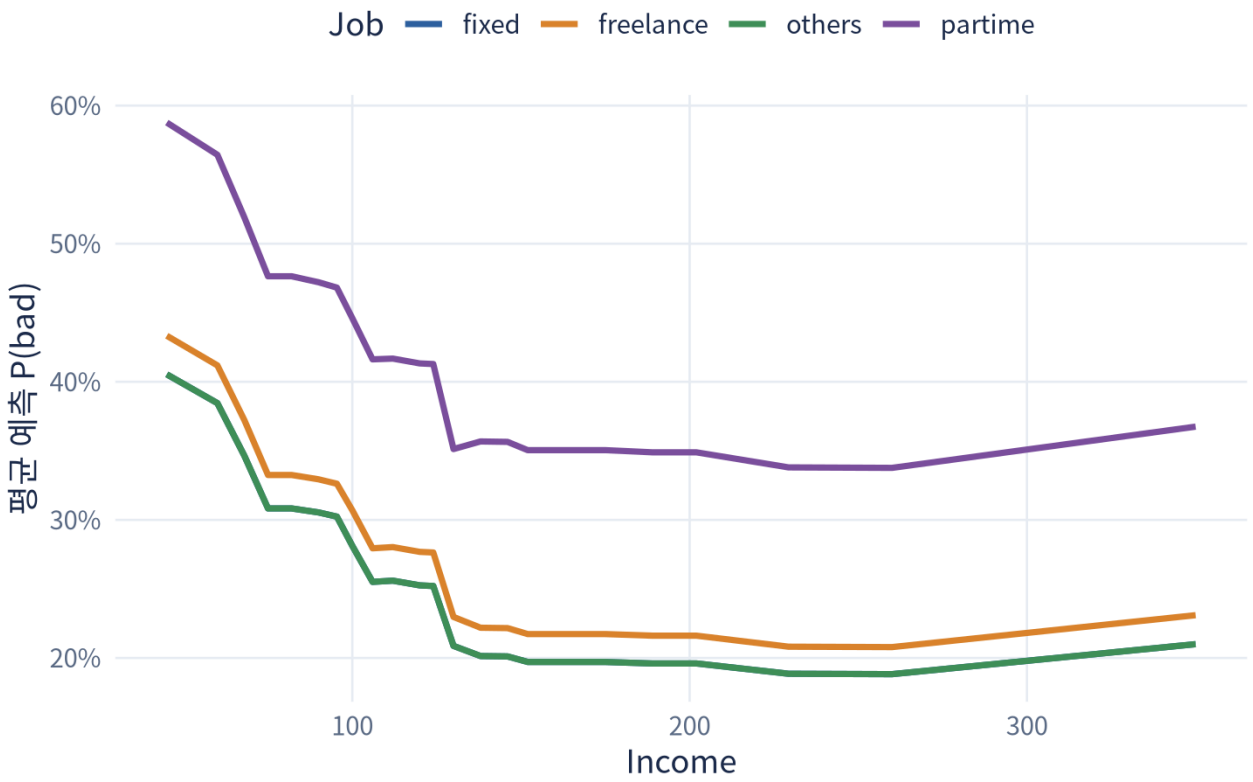
- 연체 기록 없음 : −0.23 (평균)
- 연체 기록 있음 : −0.38 (평균)
- 굵은 선 = 집단별 c-ICE 평균

해석

기록이 있는 사람은 출발점 (위험) 이 높아 줄어들 여지도 크다 . 확률 척도의 상호작용은 이런 "천장 · 바닥 효과" 일 수도 있다 (로그오즈 척도에서는 더 작을 수 있음)

학습 데이터 250 명의 ICE, XGBoost. 근속 0 년의 예측을 0 으로 맞춘 중심화 ICE

Income × Job: 고용 형태에 따라 높이가 다르다



partime 은 소득이 높아도 위험

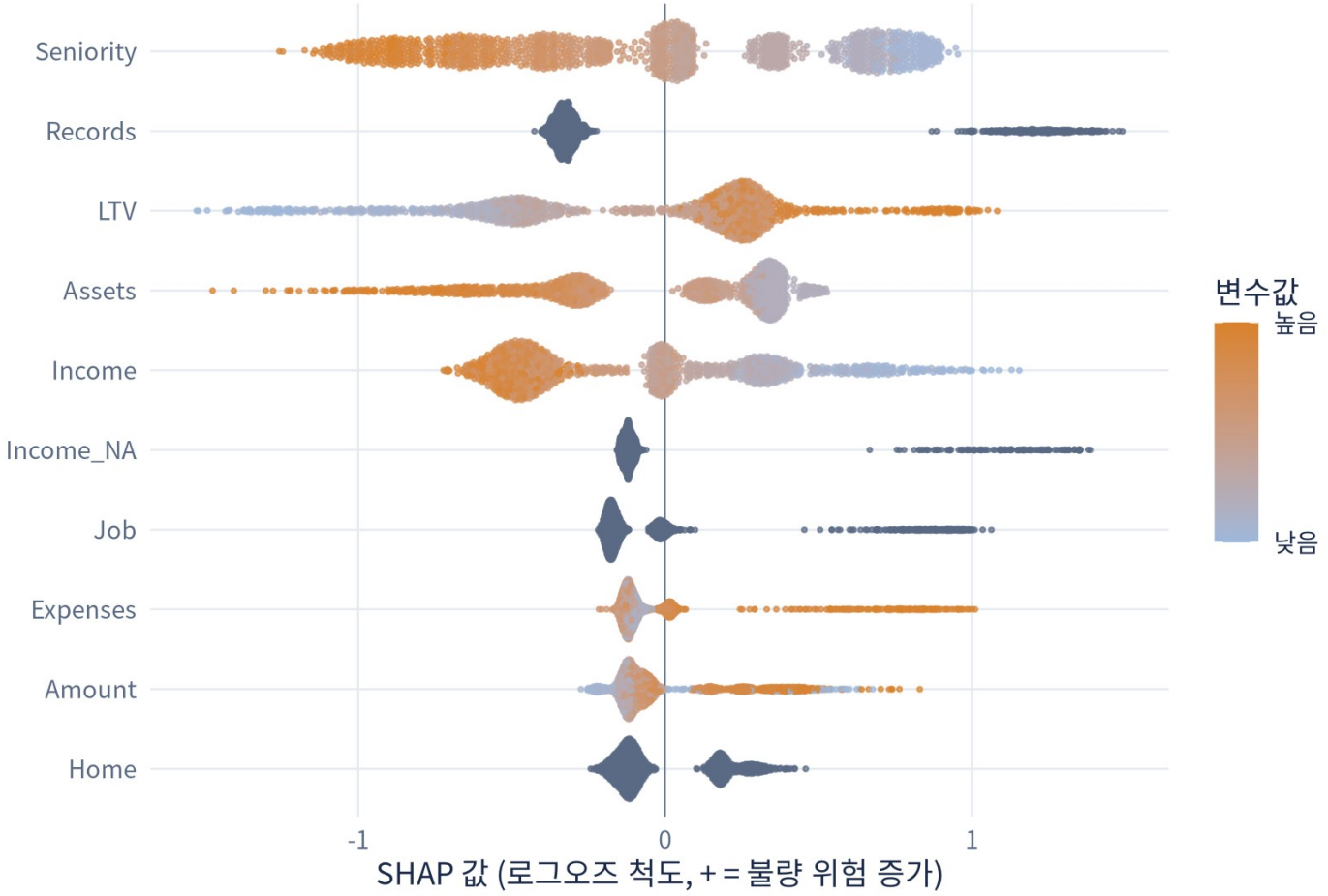
소득 200 에서 partime 35%, fixed 22%. 소득이 늘면 모두 감소하지만 partime 은 약 34% 아래로 내려가지 않는다

fixed 와 others 가 겹친다

XGBoost 는 others(171 명) 를 fixed 와 구분하지 않았다 . 표본이 작은 범주는 모형이 무시하기도 한다

곡선 모양은 비슷 , 높이만 다르다 → Income × Job 의 상호작용은 약하다 (Job 의 전체 H 도 0.08). 주로 가법적 효과

SHAP (XGBoost): 방향과 퍼짐을 한 번에



mean |SHAP| 상위 (로그오즈)

- Seniority 0.50 · Records 0.47 · LTV 0.44
- Assets 0.37 · Income 0.36
- 순열 중요도와 같은 변수들

읽는 법

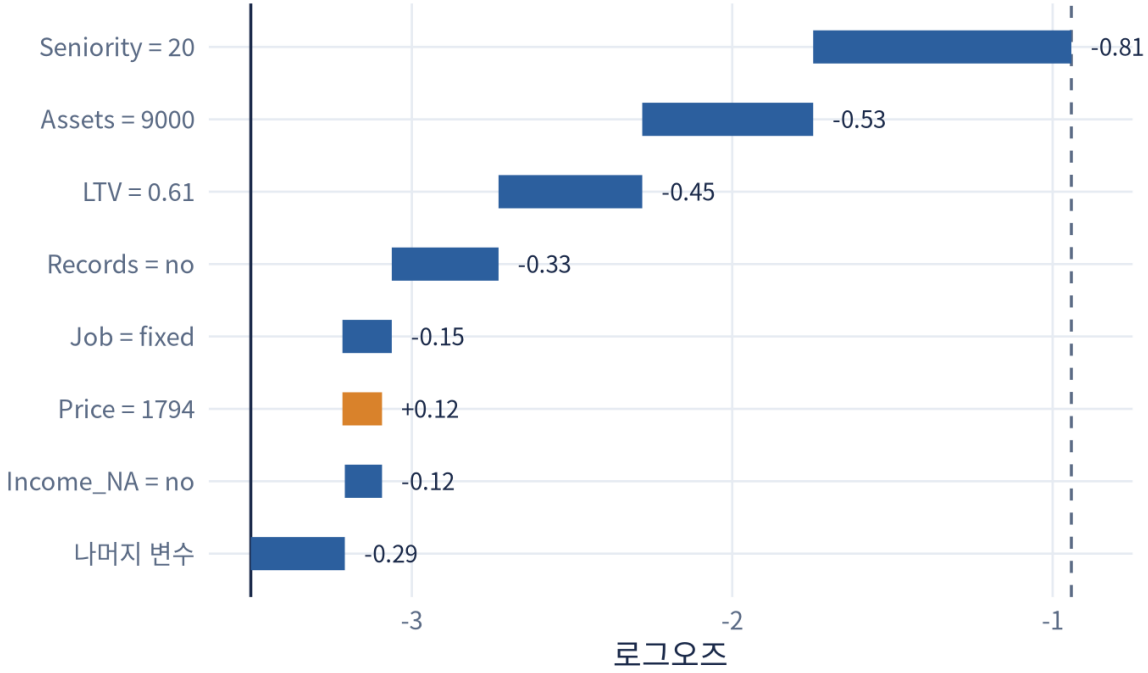
- 근속 · 자산 · 소득이 높으면 (주황) 왼쪽 (위험↓)
- LTV 가 높으면 오른쪽 (위험↑)
- Records · Income_NA = yes 는 오른쪽 끝에 따로 모인 무리 (회색 = 범주형)

테스트 1,336 명의 TreeSHAP. 더미 변수는 원래 변수로 합침 (shapviz 의 collapse). 값은 로그오즈 척도

두 신청자의 결정을 설명하기

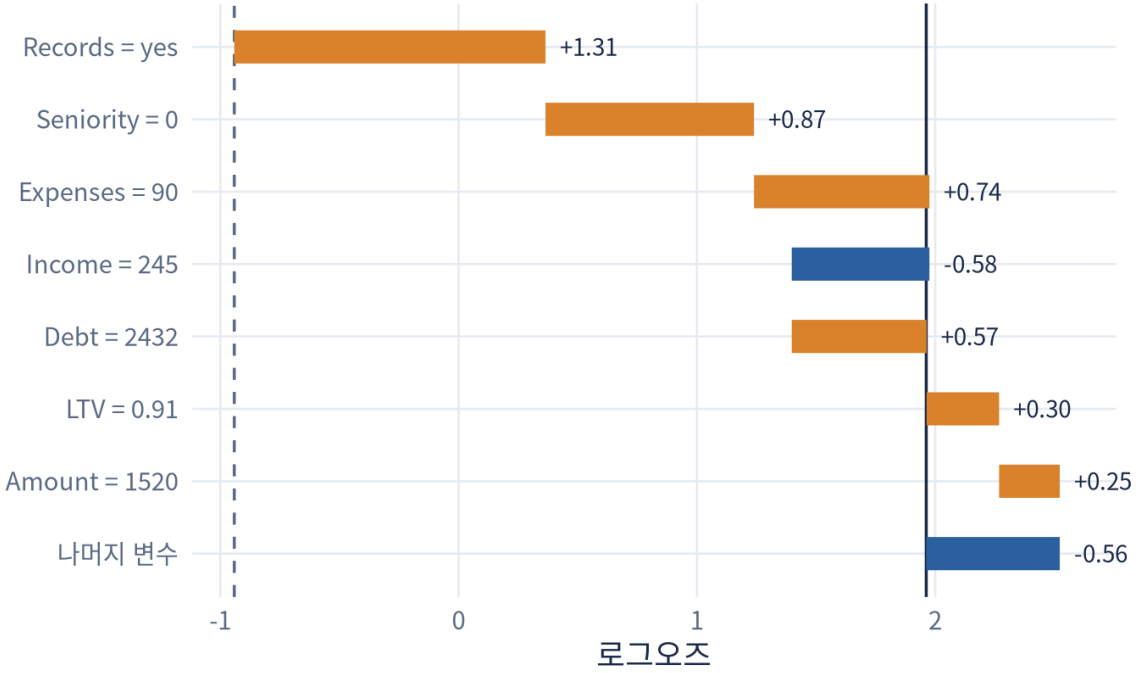
신청자 A: 낮은 위험

기준 로그오즈 -0.94 (확률 28%) → -3.50 (확률 3%)



신청자 B: 높은 위험

기준 로그오즈 -0.94 (확률 28%) → 1.96 (확률 88%)



A: 예측 3% → 승인 . 근속 20 년 , 자산 9,000, LTV 0.61

B: 예측 88% → 거절 . 연체 기록 , 근속 0 년 , 지출 90

RESPONSIBILITY

설명할 의무와 공정성

1 거절 사유 통지

많은 나라에서 대출을 거절하면 주요 사유를 알려야 한다 . B 에게 : “ 과거 연체 기록 , 짧은 근속 기간 , 높은 지출” . SHAP 상위 기여 변수가 그 근거가 될 수 있다

2 변수 사용의 제한

나이 · 혼인 상태처럼 보호되는 속성은 사용이 제한될 수 있다 . 이 모형에서 Age(AUC 감소 0.002) 와 Marital(0.001) 은 거의 기여하지 않으므로 빼도 성능 손실이 거의 없다

3 대리변수 (proxy)

변수를 빼도 다른 변수가 그 정보를 대신할 수 있다 (예 : Seniority 는 Age 와 상관 0.51). 집단별 승인율 · 오류율을 직접 점검해야 한다

4 로지스틱의 장점

계수 하나하나를 검토 · 제약할 수 있다 (예 : 부호가 상식과 맞는지). 성능 차이가 없다면 검증과 감독 측면에서 유리하다

R: XGBoost 에 pdp · iml · shapviz 쓰기

R CONSOLE

```
mm <- function(df) model.matrix(Status ~ . - 1, data = df)

# pdp: 데이터프레임 → 행렬 변환을 pred.fun 에서
XW <- structure(list(b = xgb.fit), class = "xgbwrap")
pf <- function(object, newdata)
  mean(predict(object$b, mm(as.data.frame(newdata))))
pd <- partial(XW, pred.var = "Seniority",
              train = train, pred.fun = pf)

# iml: predict.function 지정
pfun <- function(model, newdata) {
  newdata$Status <- factor("good", c("good", "bad"))
  predict(model, mm(newdata)) }
pred <- Predictor$new(xgb.fit, data = X.sub,
                      predict.function = pfun)
Interaction$new(pred)
```

R CONSOLE

```
# SHAP: 더미를 원래 변수로 묶기
Xte <- mm(test)
coll <- list(
  Home = grep("^Home", colnames(Xte),
              value = TRUE),
  Job  = grep("^Job",  colnames(Xte),
              value = TRUE))   # ...
shp <- shapviz(xgb.fit, X_pred = Xte,
               X = test[, vars],
               collapse = coll)
sv_importance(shp, kind = "beeswarm")
sv_waterfall(shp, row_id = i)
```

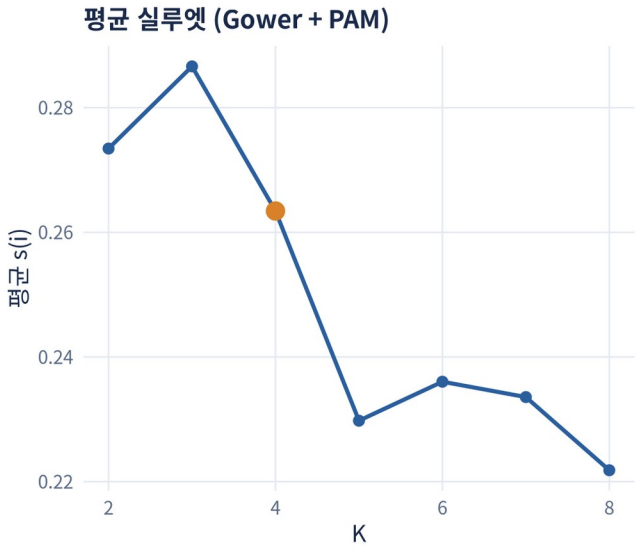
PART 7

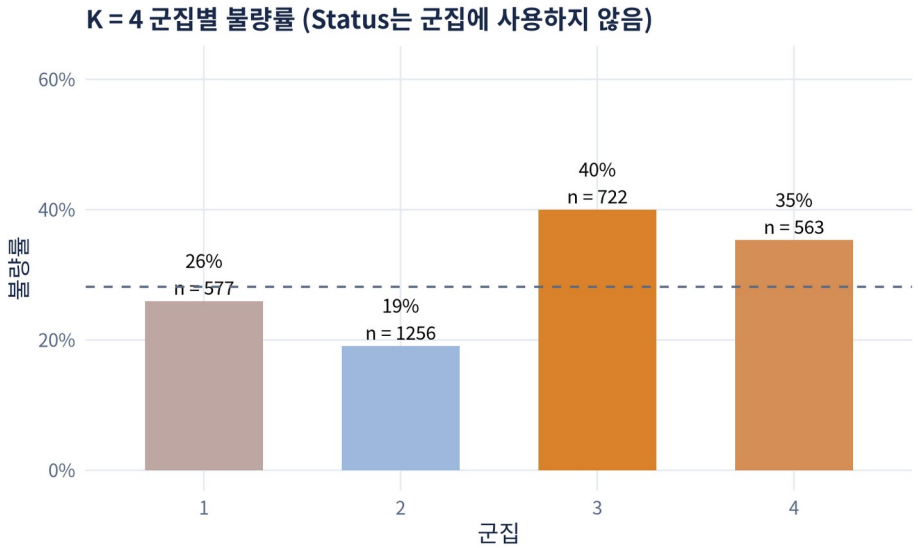
고객 세분화

Gower 거리 + PAM 으로 신청자를 묶고 , 군집별 모형이 도움이 되는지 확인한다

SEGMENTATION

섞인 변수 유형 : Gower 거리 + PAM





설정

- 신청서 변수 13 개 (Status 제외)
- 금액 변수는 log1p 후 Gower
- cluster::daisy + pam
- 실루엣 최대는 K = 3 (0.287)

K = 4 를 고른 이유 : 실루엣 차이는 작고 (0.287 vs 0.263) 해석이 더 분명하다
Status 를 전혀 쓰지 않았는데도 군집별 불량률이 19% ~ 40% 로 크게 다르다 → 군집이 위험과 관련된 구조를 잡았다 .
군집분석 강의의 메시지 그대로 : 통계 기준은 후보를 주고 , 최종 선택은 해석과 목적으로

CLUSTER PROFILES

군집 프로파일 : 이름 붙이기

군집	n	불량률	나이	근속	소득	자산	주택 소유	기혼	fixed 직업	이름 (예)
1	577	26%	43	7	124	7,000	74%	89%	0%	자가 · 비정규직 중년
2	1,256	19%	38	8	130	4,500	84%	89%	88%	자가 · 정규직 (안정형)
3	722	40%	36	5	124	0	0%	87%	73%	무주택 · 무자산 기혼
4	563	35%	25	2	102	0	0%	7%	62%	청년 · 미혼 · 사회초년

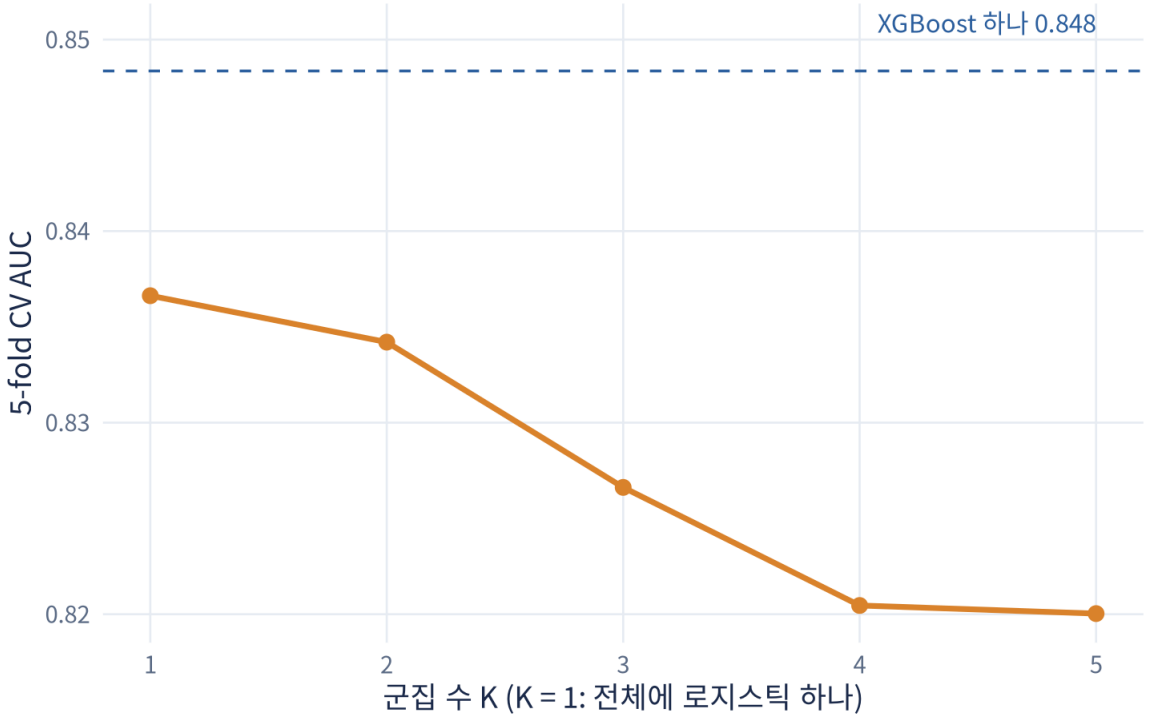
활용

- 군집별 상품 · 마케팅 (예 : 청년층에 소액 · 단기 상품)
- 현업과 대화하는 언어 : “3 번 군집의 불량률이 40%”

주의

- 중앙값 · 비율로 요약한 것 — 군집 안에도 다양한 사람이 있다
- 이름은 분석가가 붙인 것 . 현업 검토로 확정

군집별 모형은 예측을 개선했나 ? → 이번에는 아니다



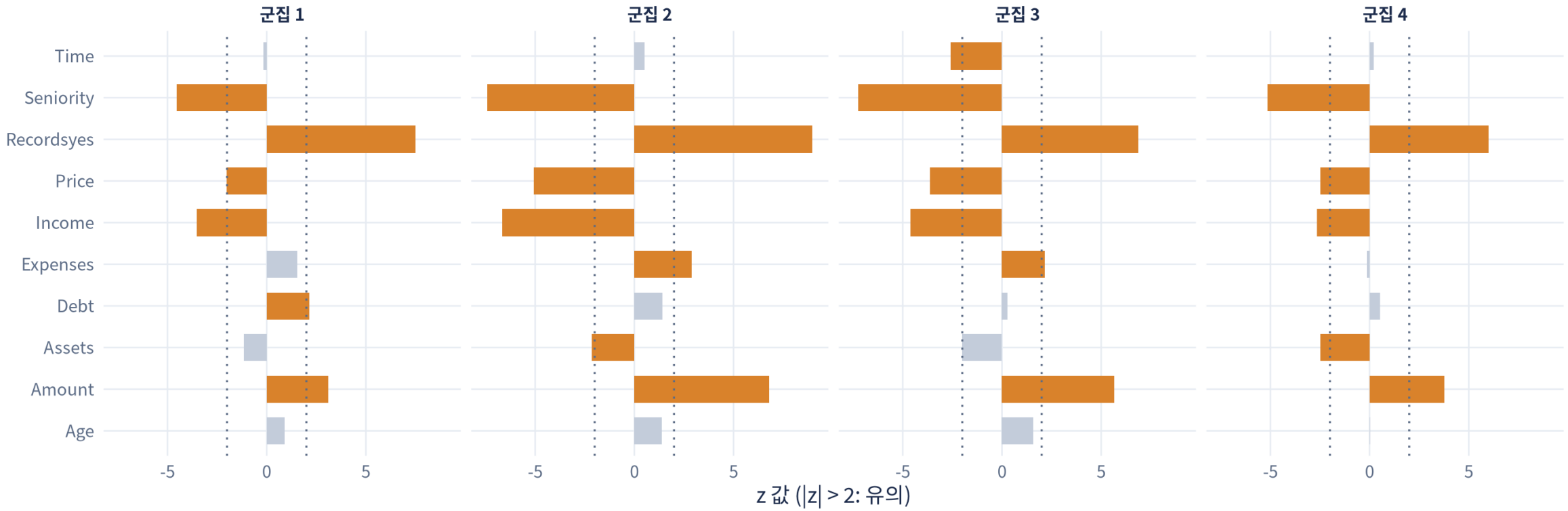
실험

- K = 1: 전체에 로지스틱 하나
- K ≥ 2: fold 마다 학습 부분으로 Gower + PAM → 군집별 로지스틱 , 검증 부분은 가장 가까운 medoid 의 모형으로 예측
- 같은 5 개 fold, 지표는 CV AUC
- K = 2: 0.834, K = 3: 0.827, K = 5: 0.820

K = 1(0.837) → K = 4(0.820): 군집을 나눌수록 나빠졌다
위험 요인이 공통이라 나눌 이유가 없고 , 군집마다 데이터가 줄어 추정이 불안정해진다 . 군집 강의의 시뮬레이션과 반대 결과 — 항상 CV 로 확인해야 하는 이유

CLUSTER-WISE COEFFICIENTS

군집마다 위험 요인이 다른가 ? (군집별 로지스틱 , z 값)



공통 : Seniority(−) · Income(−) · Records(+) · Amount(+) 는 네 군집 모두 |z| > 2. **차이 :** Expenses 는 군집 2 · 3 에서만 , Time 은 군집 3 에서만 , Debt 는 군집 1 에서만 유의 . 군집 강의의 시뮬레이션처럼 “군집마다 전혀 다른 변수” 가 중요해지는 구조는 아니다

R: Gower + PAM, 그리고 fold 안에서 군집하기

R CONSOLE

```
library(cluster)
vars <- c("Seniority","Home","Time","Age","Marital",
          "Records","Job","Expenses","Income",
          "Assets","Debt","Amount","Price")
gx <- function(df) {             # 금액은 log1p
  z <- df[, vars]
  for (v in c("Income","Assets","Debt","Amount","Price"))
    z[[v]] <- log1p(z[[v]])
  z }
D  <- daisy(gx(train), metric = "gower")
sapply(2:8, function(k)
  pam(D, k, diss = TRUE)$silinfo$avg.width)
pm <- pam(D, k = 4, diss = TRUE)
tapply(train$Status == "bad", pm$clustering, mean)
```

daisy(…, "gower")

수치형은 범위로 나눈 차이 , 범주형은 같으면 0 다르면 1 → 평균

pam(D, diss = TRUE)

거리 행렬을 직접 넣는다 . medoid = 실제 신청자

새 신청자 배정

학습 medoid 들과의 Gower 거리 중 가장 가까운 군집

CV 안에서 군집

fold 마다 학습 부분으로 pam → 테스트 부분은 medoid 로 배정 (누수 방지)

PART 8

결론과 보고

어떤 모형을 쓰자고 할 것인가 , 무엇을 조심해야 하는가

RECOMMENDATION

최종 추천 : 무엇을 기준으로 고르나

기준	로지스틱 회귀	XGBoost
순위 능력 (테스트 AUC)	0.844	0.846 (차이 유의하지 않음 , p = 0.67)
CV AUC	0.837	0.848 (5 개 fold 모두 우세)
보정 (Brier)	0.1369	0.1368
기대 비용 (CV 임계값)	0.460	0.459
설명 · 검증	계수 = 오즈비 , 제약 · 점수표로 변환 쉬움	SHAP 등 별도 도구 필요
운영	식 하나 , 어디서나 계산	모형 파일 · 라이브러리 버전 관리

추천 : 로지스틱 회귀를 주 모형 (champion), XGBoost 를 도전 모형 (challenger) 으로
성능은 사실상 같고 설명 · 검증 · 운영이 쉽다 . XGBoost 는 병행 운영하며 새 데이터에서 차이가 커지는지 지켜본다 . 결정 규칙 (임계값 0.16~0.19) 이 성능 차이보다 훨씬 중요

참고 : 로지스틱 회귀를 신용점수로 바꾸기

$$\text{점수} = \text{Offset} + \text{Factor} \cdot \ln(\text{good 오즈}), \quad \text{Factor} = \text{PDO} / \ln 2, \quad \text{Offset} = \text{기준점수} - \text{Factor} \cdot \ln(\text{기준 오즈})$$

예 : 600 점 = 오즈 50 : 1, PDO = 20

- PDO: 오즈가 2 배가 되는 데 필요한 점수 (Points to Double the Odds)
- Factor = 20 / ln 2 = 28.85
- Offset = 600 − 28.85 · ln 50 = 487.1

신청자	로지스틱 P(bad)	good 오즈	점수
A	0.022	44.2 : 1	596
B	0.806	0.24 : 1	446

점수가 20 점 높을 때마다 good 오즈가 2 배

왜 점수로 바꾸나 ?
현업은 확률보다 점수에 익숙하고 , 로지스틱은 로그오즈가 변수별 기여의 합이라 “변수별 점수표” 로 나눌 수 있다 (근속 0~2 년 : −15 점 같은 식).
수업에서는 개념만 — 시간이 없으면 건너뛰어도 됨

CHECKLIST

이번 분석에서 확인한 함정 체크리스트

✓ **누수**

신청 시점 이후 정보 없음 ? 전처리는 분할 후 학습 데이터로 ?

✓ **결측의 의미**

무작위 결측인가 ? Income 결측 = 불량률 57%

✓ **이상한 값**

Expenses = 35, Assets = 0 의 의미를 현업에 확인했나 ?

✓ **정확도의 함정**

불균형 자료에서 정확도 대신 AUC · 비용

✓ **공정한 비교**

같은 fold, 튜닝도 CV 안에서 , 테스트는 한 번

✓ **작은 차이**

CV 표준편차 · 신뢰구간 · DeLong 으로 확인

✓ **임계값**

0.5 가 아니라 비용 · 업무 제약으로

✓ **해석의 한계**

상관된 변수 , 인과 아님 , 보호 속성 · 대리변수

AFTER DEPLOYMENT

모형을 배포한 뒤에 할 일

1 모니터링

신청자 분포가 바뀌는지 (PSI 등), 실제 불량률이 예측과 맞는지 매달 확인. 경기 변화에 모형은 쉽게 낡는다

2 Reject inference

거절된 신청자의 결과는 영원히 모른다 → 일부를 무작위로 승인하거나 외부 신용정보로 보완하는 방법

3 재학습 주기

새 데이터가 쌓이면 같은 절차 (분할 → 전처리 → CV) 로 다시. 이 강의의 코드가 그대로 파이프라인이 된다

4 데이터를 더 모으기

모형을 바꿔서 얻은 AUC 개선은 0.01 수준. 신용조회 정보, 거래 내역처럼 새로운 변수가 훨씬 큰 차이를 만든다

5 보고서

목적 · 데이터 한계 · 성능 (신뢰구간 포함) · 결정 규칙 · 주요 위험 요인 · 공정성 점검 · 모니터링 계획을 한 문서로

SUMMARY

오늘의 핵심 정리

1	문제 정의	사건 · 비용 (5 : 1) · 평가 기준 (AUC, 비용 , 보정 , 설명) 을 모형보다 먼저 정한다
2	데이터	Income 결측은 불량률 57% → 지시변수로 보존 . 분할 후 학습 데이터로만 전처리
3	모형	로지스틱 0.844 ≈ RF 0.842 ≈ XGBoost 0.846 (테스트 AUC, 차이 유의하지 않음). 단일 트리 0.753
4	결정	임계값 0.5 → 0.16 으로 기대 비용 36% 감소 . 모형 선택보다 결정 규칙의 효과가 크다
5	해석	연체 기록 · 근속 · LTV · 자산 · 소득이 핵심 . Records 가 다른 변수의 효과를 바꾼다
6	세분화	군집은 위험 구조를 잘 요약했지만 , 군집별 모형은 예측을 개선하지 못했다 → CV 로 확인

좋은 분석 = 좋은 모형 + 좋은 질문 + 정직한 평가 + 설명할 수 있는 결론